Enterprise Agent Design Patterns

Patterns, practices, and lessons for building AI agent systems

A practitioner's guide — from single agents to multi-agent architectures

March 2026

Hanuma Pedprolu

M.Tech AI ML | BITS Pilani (WILP)

About Me

Hanuma Pedprolu

AI Architect • M.Tech AI ML(25-27, pursuing), BITS Pilani (WILP)

AI Architect specialising in Generative AI and Agentic AI systems, with 15 years of hands-on experience building software at scale.

Started as a software developer, moved into consulting, grew into architecture — and have been working across various flavours of AI since 2019, from classical ML to LLMs to multi-agent systems.

2011	2015	2017	2019	2022	2024
Software Developer	Consultant	Architect	AI / ML	Generative AI	Agentic AI

Webinar Agenda

Here's what we'll walk through together

01 Foundation & Taxonomy

Classification, maturity model, agent spectrum

02 Single Agent Patterns

ReAct, CodeAct, ReWOO, LATS, Plan-Execute, Reflexion

03 Multi-Agent Patterns

Supervisor, Hierarchy, Peer, MoA, Handoff, Sub-Agents

04 Swarm Patterns

Emergent, Stigmergy, Orchestrated vs Emergent

05 Hybrid Patterns

10+ real-world hybrid architectures + Design Canvas

06 Tool & Capability Patterns

RAG, Agentic RAG, Deep Research, Memory, Caching

07 Orchestration & Communication

Router, DAG, MCP/A2A, Context Mgmt, Error Handling

08 Engineering Principles

SOLID, Microservices, 12-Factor, Prompt Mgmt, Model Routing

09 Enterprise Governance

Security, OpenTelemetry, Arize AI, Elastic, Guardrails

What's In It For You?

Whether you're building, architecting, or leading — here's what you'll walk away with

For Engineers & Developers

- Know exactly when to use ReAct vs CodeAct vs ReWOO vs Plan-Execute
- Build agents that self-debug, self-correct, and manage their own memory
- Implement RAG, tool calling, and code execution patterns correctly
- Avoid the top 10 anti-patterns that kill production agents
- Copy-paste the I/O contracts and specs bars for your own designs

For Solution Architects

- Choose the right agent topology: single, multi, swarm, or hybrid
- Map SOLID, microservices, and 12-Factor principles to agent systems
- Design sub-agent hierarchies with proper communication protocols
- Build observability into hybrid systems from day one (OTel, Arize)
- Use the Hybrid Design Canvas to compose production architectures

For Tech Leads & Engineering Managers

- Govern agent costs: model routing saves 40-70% without quality loss
- Set up guardrails, approval gates, and human-in-the-loop patterns
- Evaluate frameworks: LangGraph vs CrewAI vs AutoGen vs Semantic Kernel
- Build evaluation & testing pipelines that catch regressions before prod
- Implement security patterns against prompt injection and data exfiltration

For Enterprise & Platform Teams

- Agent-as-a-Service: build a reusable platform, not one-off agents
- Cost attribution, budget controls, and chargeback models for AI spend
- Compliance-ready: audit trails, data residency, model governance
- Decision matrix: match patterns to business use cases systematically
- Real case studies: customer support agent, code execution from UI

5% AI. 100% Engineering.

"Building AI agents demands governed data pipelines, guardrails, monitoring, and ACL-aware retrieval — 5% AI, 100% engineering discipline."

— MarkTechPost, 2025
(https://www.marktechpost.com/2025/09/18/building-ai-agents-is-5-ai-and-100-software-engineering/)

Why this deck exists

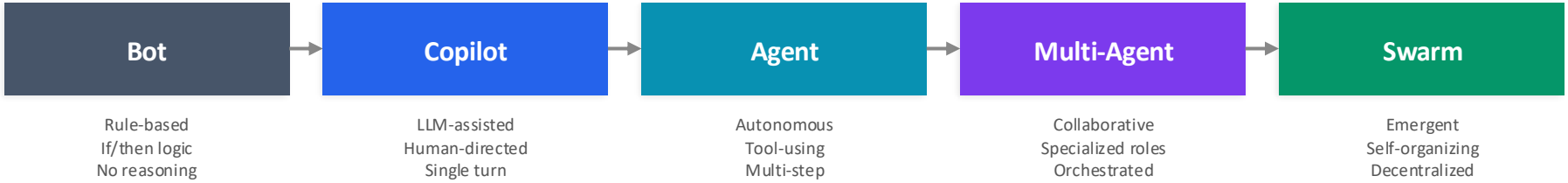
The LLM is the easy part. The hard part is everything around it — orchestration, memory management, tool design, error handling, security, observability, cost control, and governance.
This webinar gives you the engineering patterns to build the other 95%.

01 Foundation & Taxonomy

Classifying the agent landscape — from bots to swarms

What is an Agent?

The definition spectrum — from simple automation to autonomous intelligence



Core Agent Characteristics

- Perception — observes environment via inputs/tools
- Reasoning — plans, decides, reflects on actions
- Action — executes via tools, APIs, code
- Memory — maintains state across interactions
- Autonomy — operates with minimal human guidance

Enterprise Requirements

- Determinism — predictable, auditable behavior
- Guardrails — safety boundaries and approval gates
- Observability — full trace of reasoning + actions
- Scalability — handles enterprise workloads
- Governance — compliance, cost control, access mgmt

Agent Taxonomy

Master classification framework — three dimensions of agent design

Dimension 1: Control Model

DETERMINISTIC

Fixed rules, if/then, state machines, DAGs

PROBABILISTIC

LLM-driven decisions, adaptive, creative

→ *Spectrum* ←

Dimension 2: Cardinality

SINGLE

One agent, one task

MULTI

2-10 agents, coordinated

SWARM

10+ agents, emergent

Dimension 3: Behavior Model

REACTIVE

Responds to input only

PROACTIVE

Initiates actions autonomously

DELIBERATIVE

Plans before acting

REFLEXIVE

Learns from own output

Agent Maturity Model

L0 → L5: From simple automation to fully autonomous swarms

L0	**Rule Engine** If/then automation, no LLM	**Memory:** None	**Runtime:** <100ms
L1	**LLM Router** LLM classifies → routes to code	**Memory:** Stateless	**Runtime:** 1-3s
L2	**Tool Agent** LLM reasons + calls tools	**Memory:** Context window	**Runtime:** 5-30s
L3	**Planning Agent** Multi-step planning, reflection	**Memory:** Short-term + episodic	**Runtime:** 30s-5m
L4	**Multi-Agent** Coordinated specialized agents	**Memory:** Shared + individual	**Runtime:** 1-30m
L5	**Autonomous Swarm** Self-organizing, emergent behavior	**Memory:** Distributed + persistent	**Runtime:** Hours+

02 Single Agent Patterns

The foundational building blocks — one agent, one task, many approaches

Single Agent Patterns

Comprehensive view — four core patterns and when to use each

ReAct

Interleaved reasoning and acting. Think → Act → Observe loop.

Use: General-purpose, web browsing, research

MEM: Context window

RUNTIME: 5-30s

Hybridizes with →

RAG

Tool-Use

LLM selects and invokes tools based on user intent.

Use: API orchestration, data lookup, CRUD

MEM: Stateless / session

RUNTIME: 2-15s

Hybridizes with →

Tool-Use

Plan & Execute

Creates full plan first, then executes step by step.

Use: Complex multi-step tasks, code generation

MEM: Plan state + context

RUNTIME: 30s-5m

Hybridizes with →

Multi-Agent

Reflexion

Self-evaluates output, critiques, then revises.

Use: Code debugging, writing, quality-critical

MEM: Episodic (past attempts)

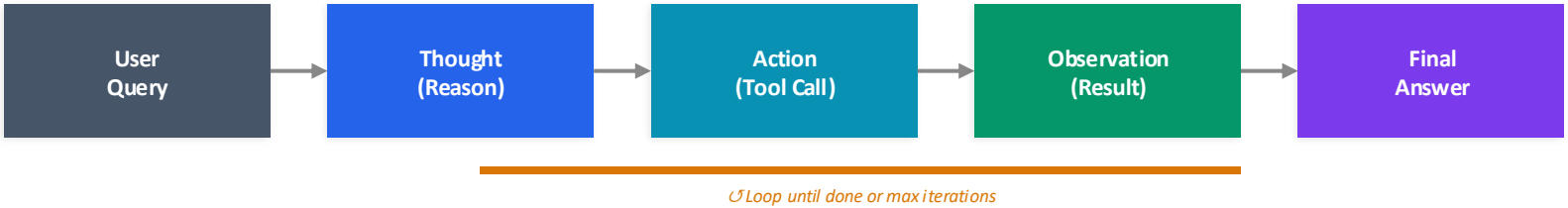
RUNTIME: 1-10m

Hybridizes with →

Memory

ReAct Pattern — Double-Click

Reason + Act: The thinking-acting loop that powers most modern agents



MEMORY	RUNTIME	TOKEN COST	DETERMINISM
Context Window (grows per loop)	**5-30s (2-8 loops typical)**	**Medium-High (cumulative)**	**Low — LLM decides each step**

Strengths

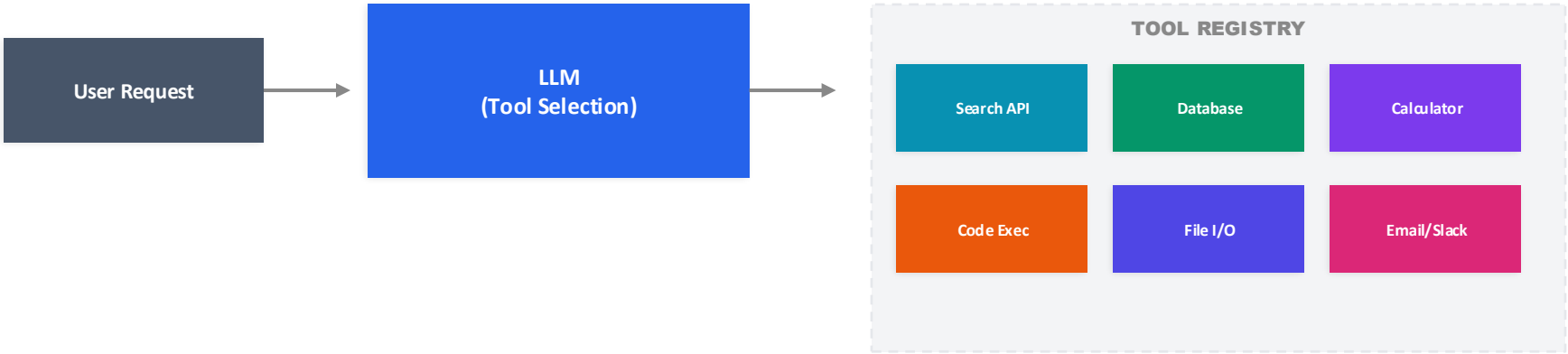
- Highly flexible — adapts to any task shape
- Transparent reasoning — easy to trace decisions
- Self-correcting — can retry on observation errors
- Works with any tool set
- Most widely supported (LangChain, CrewAI, etc.)

Weaknesses + Enterprise Risks

- Token cost grows linearly per loop iteration
- No upfront planning — can go in circles
- Context window overflow on complex tasks
- Hard to set deterministic time/cost bounds
- Requires robust tool error handling

Tool-Use Agent Pattern — Double-Click

LLM as intelligent function dispatcher — the backbone of enterprise agents



MEMORY	RUNTIME	TOKEN COST	DETERMINISM
Stateless / Session	**2-15s per call**	**Low-Medium**	**Medium — LLM picks tool**

Design Principles

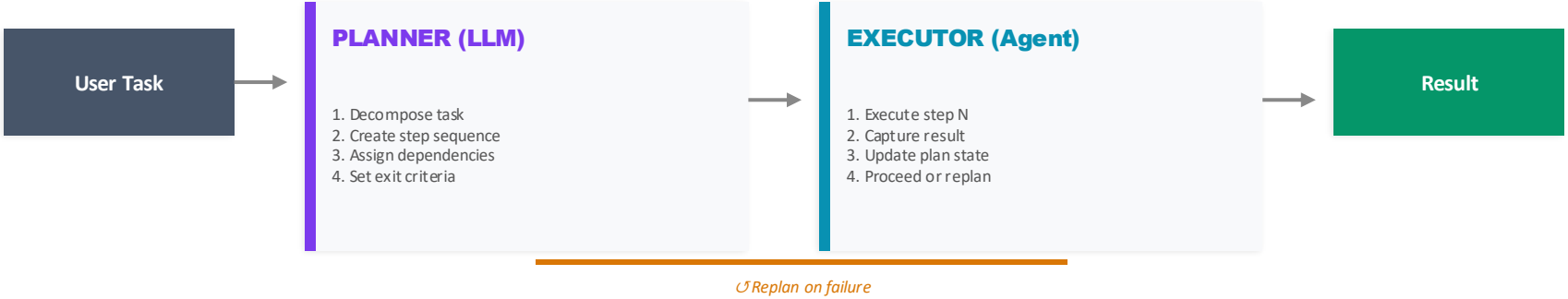
- Schema-first: Define tools with JSON schema / OpenAPI
- Single Responsibility: One tool = one capability
- Idempotent tools for safe retries
- Validation layer between LLM and tool execution

Enterprise Patterns

- MCP (Model Context Protocol) for tool discovery
- Tool authorization with scoped permissions
- Rate limiting and cost caps per tool
- Audit log every tool invocation for compliance

Plan & Execute Pattern — Double-Click

Think first, act second — separating planning from execution for complex tasks



MEMORY	RUNTIME	TOKEN COST	DETERMINISM
Plan State + Step Results	**30s-5min (plan-dependent)**	**High (planning overhead)**	**Medium-High (plan is fixed)**

When to Use

- Complex, multi-step tasks with clear subtasks
- When you need predictable execution order
- Code generation + testing workflows
- Data pipeline orchestration
- Tasks where partial failure is acceptable

Gotchas & Trade-offs

- Planning LLM call adds latency + cost upfront
- Plan may be wrong — replanning is expensive
- Not great for highly dynamic environments
- Executor still needs ReAct-like capability
- SOLID: Open/Closed — extend plan types, don't modify planner

Reflexion / Self-Critique Pattern — Double-Click

Learn from mistakes — agents that evaluate and improve their own output

MEMORY	RUNTIME	TOKEN COST	DETERMINISM
Episodic (past attempts + scores)	1-10 min (N iterations)	High (N× base cost)	Low — each iteration varies

Best Practices

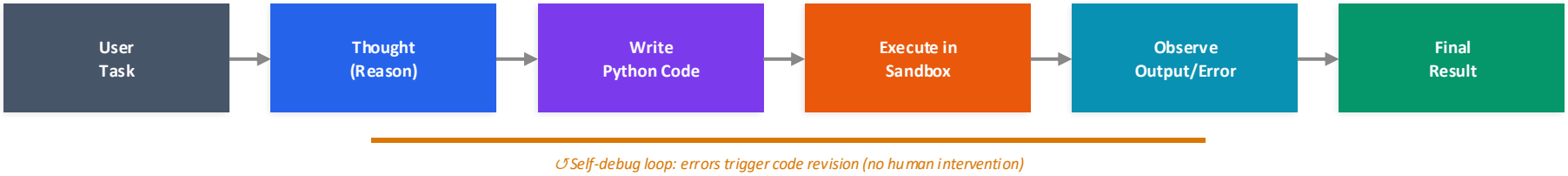
- Set hard iteration cap (3-5 rounds max)
- Use structured scoring rubric, not vibes
- Store episodic memory for cross-session learning
- Separate evaluator from generator (avoid bias)
- Log all attempts for human review
- Consider cost-quality trade-off per use case

Enterprise Applications

- Code generation → test → fix → retest cycles
- Document drafting with compliance checks
- SQL query optimization with explain plan validation
- Security audit reports with automated verification
- Content generation meeting brand guidelines
- Hybrid: Reflexion + RAG for grounded self-improvement

CodeAct Pattern — Double-Click

Code as action — the agent writes and executes Python instead of calling predefined tools



KEY DIFFERENCE: ReAct calls predefined tools via JSON. CodeAct **writes executable code** — the action space is unlimited. Agent can compose tools, create new functions, and self-debug. Powers Claude Code, Cursor, OpenHands.

MEMORY	RUNTIME	TOKEN COST	DETERMINISM
Code state + execution context	**5-60s (execution-dependent)**	**30% fewer steps than JSON**	**Medium — code is reproducible**

Why CodeAct Wins

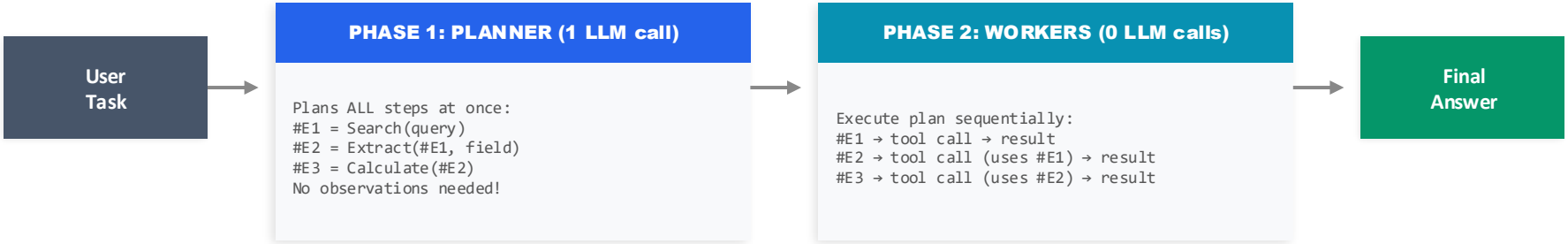
- Unlimited action space — not constrained by predefined tools
- Compose tools dynamically: combine APIs in one code block
- Self-debugging: errors auto-trigger code revision
- 30% fewer LLM calls → 30% cheaper than JSON/text actions
- Leverages LLM's pre-training on code data

Enterprise Risks & Mitigations

- CRITICAL: Arbitrary code execution requires sandboxing
- Docker/Firecracker isolation is mandatory in production
- Resource limits: CPU, memory, network, execution time
- Code review layer: validate before execution for risky ops
- Audit trail: log every code block generated and executed

ReWOO Pattern — Double-Click

Reasoning WithOut Observations — plan all steps upfront, execute without mid-loop LLM calls



ReAct: N LLM calls (1 per loop) | ReWOO: 1 LLM call (plan only) | CodeAct: N LLM calls but 30% fewer steps

MEMORY	RUNTIME	TOKEN COST	DETERMINISM
Plan state only (small)	**Fast — no mid-loop LLM waits**	**Lowest — single LLM call**	**High — plan is fixed**

When to Use ReWOO

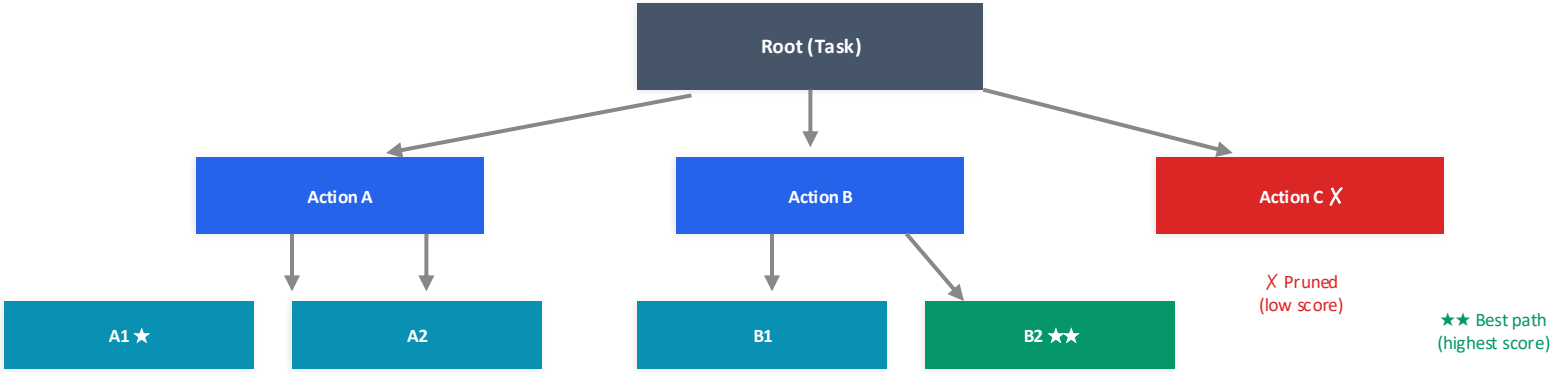
- Predictable, structured tasks with known tool sequence
- Batch processing where speed matters more than adaptability
- Cost-sensitive environments (1 LLM call vs N)
- Report generation, data extraction, known pipelines
- High throughput applications requiring parallelizable execution

Critical Trade-off: Fragility

- No self-correction: if plan is wrong, entire execution fails
- Cannot adapt to unexpected tool responses mid-execution
- Not suitable for dynamic or exploratory tasks
- Mitigation: validate plan before execution, add fallback to ReAct
- Hybrid: ReWOO for fast path, ReAct as fallback on error

LATS — Language Agent Tree Search

Monte Carlo tree search for agents — explore, evaluate, backtrack, and find optimal action paths



MEMORY	RUNTIME	TOKEN COST	QUALITY
Full tree state (high)	**Minutes-hours (exploration)**	**Very High (N paths × depth)**	**Highest — optimal path found**

How LATS Works

- Selection: pick most promising node (UCB score)
- Expansion: generate possible next actions
- Evaluation: score each path with LLM or heuristic
- Backpropagation: update scores up the tree
- Repeat until solution found or budget exhausted

When to Use (Rare but Powerful)

- Extremely hard reasoning tasks (math, proofs, puzzles)
- When one-shot or ReAct can't find the answer
- Competitive coding, theorem proving, complex planning
- NOT for typical enterprise tasks (too expensive)
- Consider: is 10× cost justified by quality gain?

Single Agent Comparison Matrix

When to use which pattern — trade-offs at a glance

Pattern	Determinism	Memory Load	Runtime	Token Cost	Flexibility	Best For
ReAct	Low	Medium	5-30s	Medium-High	★★★★★	General tasks
Tool-Use	Medium	Low	2-15s	Low	★★★☆☆	API orchestration
Plan & Execute	Med-High	High	30s-5m	High	★★★★☆	Complex workflows
Reflexion	Low	High	1-10m	Very High	★★★☆☆	Quality-critical

Quick Decision Guide

- Start with ReAct for prototyping — it's the most flexible and widely supported pattern
- Move to Tool-Use when you need faster, cheaper, more predictable single-shot tool calls
- Upgrade to Plan & Execute when task complexity requires upfront decomposition
- Add Reflexion when output quality matters more than speed/cost (code gen, legal docs, compliance)

03 Multi-Agent Patterns

Coordinated intelligence — specialized agents working together

Multi-Agent Patterns

Comprehensive view — five architectures for agent collaboration

Hub & Spoke

Supervisor

Central controller delegates to workers

MEM: Supervisor holds state

RT: 1-10m

Hybridizes with RAG, Tools, Memory

Tree

Hierarchical

Managers delegate to sub-managers to workers

MEM: Per-level state

RT: 5-30m

Hybridizes with RAG, Tools, Memory

Mesh

Peer-to-Peer

Agents debate, critique, and reach consensus

MEM: Shared blackboard

RT: 2-15m

Hybridizes with RAG, Tools, Memory

Fan-out/in

Mixture (MoA)

Multiple agents generate, one aggregates

MEM: Parallel contexts

RT: 1-5m

Hybridizes with RAG, Tools, Memory

API Mesh

Agent-as-a-Service

Composable agents exposed as services

MEM: Stateless / event

RT: Varies

Hybridizes with RAG, Tools, Memory

Supervisor Pattern — Double-Click

Central orchestrator delegates to specialized worker agents

MEMORY	RUNTIME	SCALING	SOLID MAPPING
Supervisor: full | Workers: scoped	**1-10 min (sequential workers)**	**Add workers without changing supervisor**	**SRP + DIP (depend on abstractions)**

Microservice Parallel

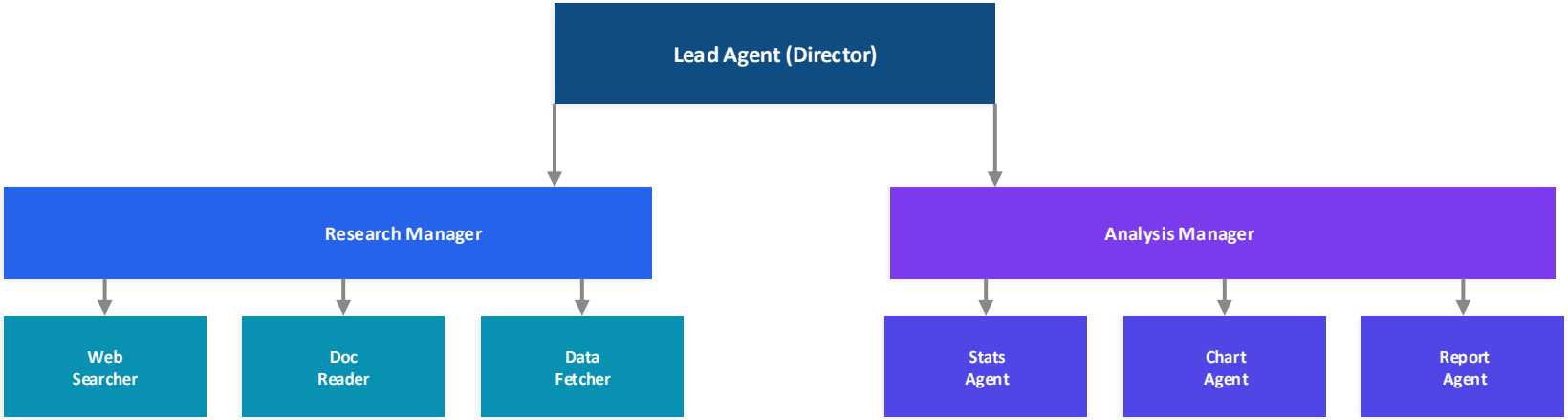
- Supervisor = API Gateway / Orchestrator
- Workers = Microservices with defined contracts
- State management = Saga pattern

Key Risk: Bottleneck

- Supervisor is single point of failure
- Mitigation: circuit breaker + fallback supervisor
- Monitor supervisor token usage closely

Hierarchical Delegation Pattern

Tree-structured management — managers delegate to sub-teams



MEMORY	RUNTIME	TOKEN COST	SOLID MAPPING
Cascading: each level scoped	**5-30 min (deep trees = slow)**	**Very High (N agents × context)**	**ISP — each level has minimal interface**

When to Use

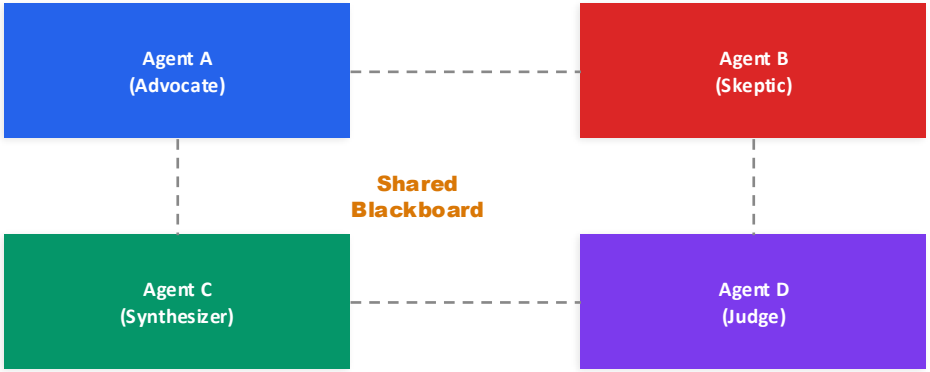
- Large-scale research with many sub-topics
- Enterprise workflows spanning multiple departments
- When tasks naturally decompose into sub-tasks
- Org-chart-style delegation mirrors business logic

Anti-Patterns to Avoid

- Too many levels (>3) — latency compounds
- Managers that just pass-through without adding value
- No feedback loop from workers to managers
- Microservice anti-pattern: distributed monolith

Peer-to-Peer / Debate & Critique Pattern

No boss — agents argue, challenge, and reach consensus



MEMORY	RUNTIME	TOKEN COST	CONSENSUS
Shared blackboard + individual	2-15 min (rounds of debate)	Very High (N agents × rounds)	Voting / Judge / Convergence

Strengths

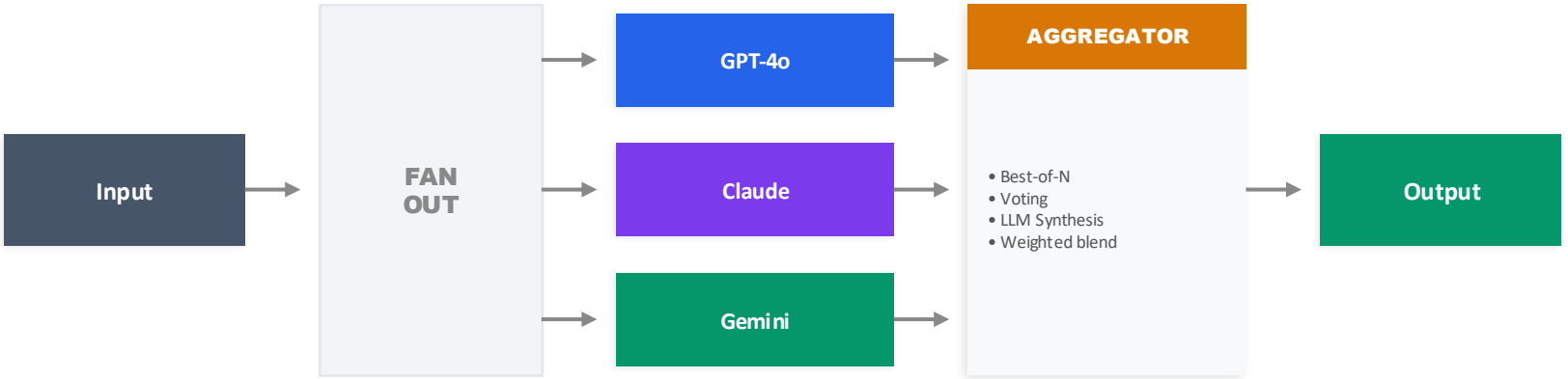
- Reduces hallucination through adversarial checking
- Multiple perspectives improve output quality
- No single point of failure
- Great for risk assessment and decision-making

Challenges

- Token cost multiplied by agents × rounds
- Can loop endlessly without convergence criteria
- Hard to debug — which agent caused the error?
- Requires careful prompt engineering per role

Mixture of Agents (MoA) Pattern

Fan-out to many, aggregate the best — ensemble intelligence



MEMORY	RUNTIME	TOKEN COST	QUALITY
Parallel isolated contexts	**Max(agents) + aggregation**	**N× single agent cost**	**Higher than any single agent**

Use Cases

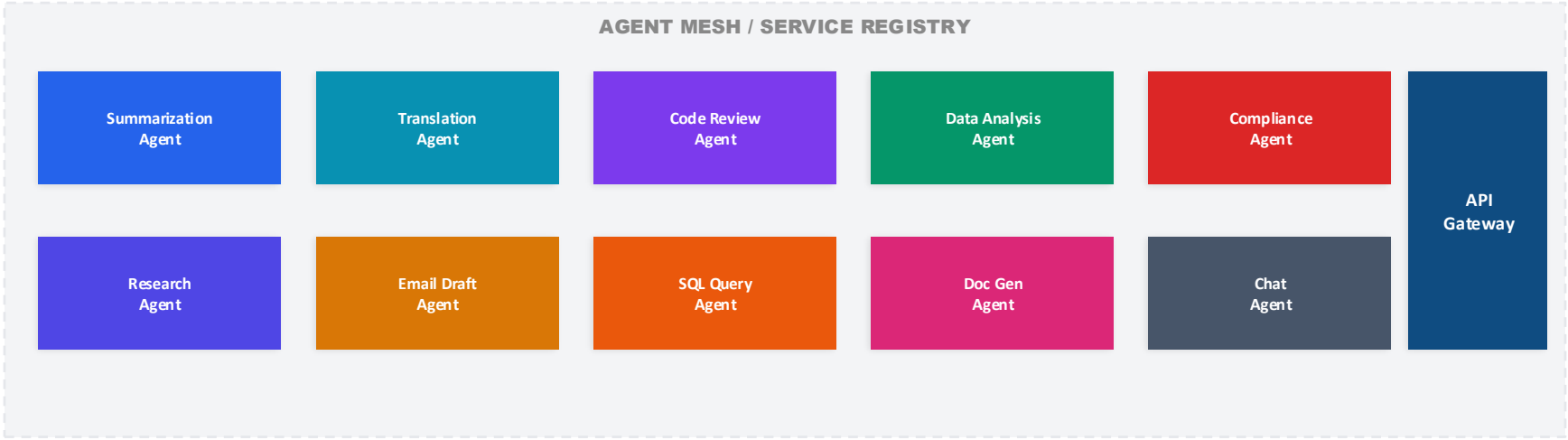
- Code generation — generate N solutions, pick best
- Research synthesis — diverse source coverage
- Translation — multiple models, best output
- Risk assessment — ensemble reduces blind spots

Design Decisions

- Same model with different prompts vs different models?
- Aggregation strategy: voting vs LLM-judged?
- Cost justification: is N× cost worth quality gain?
- Microservice parallel: Load balancer + service mesh

Agent-as-a-Service (AaaS) Pattern

Composable, API-first agents — the microservices of AI



MEMORY	RUNTIME	SCALING	SOLID
Stateless (event-driven)	**Varies per agent**	**Independent per agent**	**SRP + OCP + DIP all apply**

Microservice Mapping

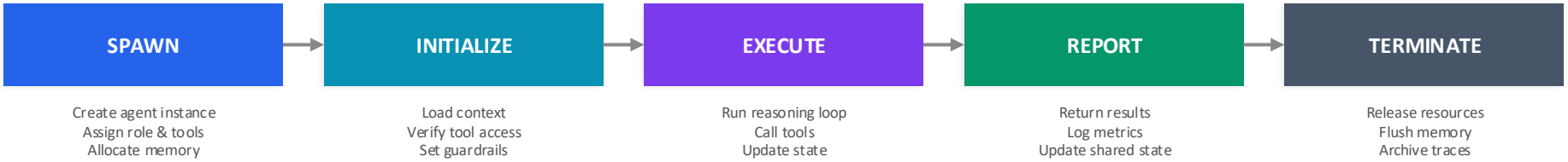
- Each agent = microservice with defined API contract
- Service discovery for agent capabilities
- Independent deployment, versioning, scaling

Enterprise Benefits

- Teams own individual agents independently
- A/B test agent implementations transparently
- Cost attribution per agent per team

Agent Lifecycle — Double-Click

Spawn → Initialize → Execute → Report → Terminate — managing agent lifecycles



Memory & Resource Timeline

Runtime Considerations

- Context window = primary memory cost (grows per step)
- Tool calls = I/O latency (network, API rate limits)
- Cold start: first agent invocation is slowest
- Warm pools: keep agents ready for hot paths
- Token budget: set hard limits per agent lifecycle

Failure Modes

- Context overflow: agent exceeds token limit mid-task
- Zombie agents: stuck in loop, never terminate
- Resource leak: memory not freed on crash
- Cascade failure: one agent crash kills the chain
- Mitigation: timeouts, circuit breakers, graceful degradation

Agent Handoff Pattern

Explicit control transfer between agents — the pattern behind OpenAI Swarm

Handoff = agent explicitly transfers control + conversation context to another agent. Only ONE agent is active at a time. Clean, auditable, no parallel confusion.

MEMORY	RUNTIME	COMPLEXITY	PATTERN
Full context passed at handoff	Sequential: sum of active agents	Low — simple state machine	Chain of Responsibility (GoF)

Design Principles

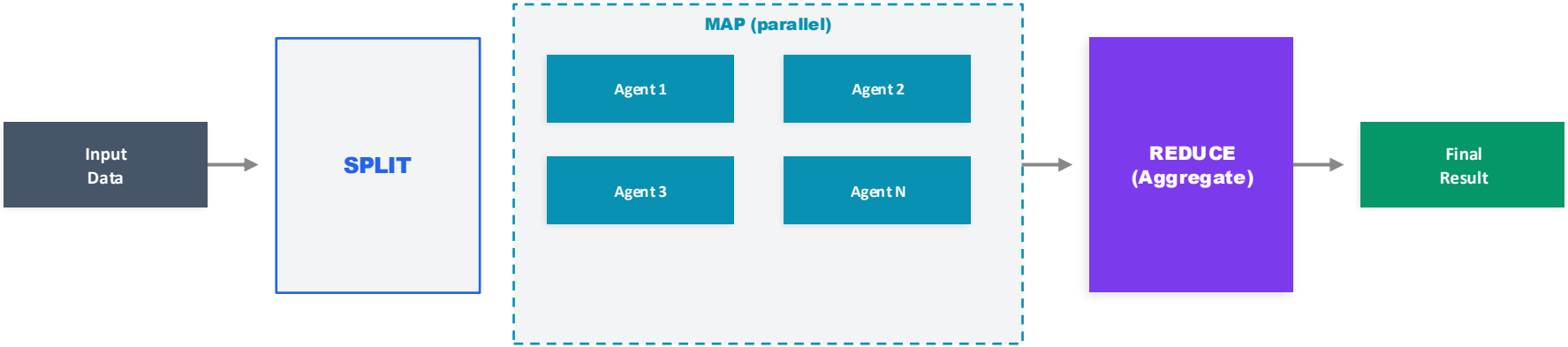
- Each agent has a clear role + handoff conditions
- Context is carried: full conversation transfers
- Only one agent active — no coordination overhead
- Handoff is explicit (function call), not implicit
- Return to previous agent is supported (backtrack)
- Maps to microservice: Chain of Responsibility pattern

Enterprise Use Cases

- Customer service: triage → sales → support → billing
- Onboarding: intake → KYC → account setup → welcome
- Incident response: detect → classify → resolve → report
- Loan processing: application → credit → underwrite → close
- Clean audit trail: every handoff logged with reasoning
- Simpler than supervisor: no central orchestrator needed

Map-Reduce for Agents

Fan-out parallel processing → aggregate results — the workhorse of scalable agent systems



MEMORY	RUNTIME	SCALING	COST
N × agent context (parallel)	O(max agent) — true parallelism	Linear with data volume	N × single agent (but faster)

Enterprise Applications

- Document analysis: split 100-page doc → parallel summarize → merge
- Code review: split repo by module → parallel review → aggregate findings
- Research: parallel search N topics → synthesize insights
- Data validation: split dataset → parallel check → merge errors
- Translation: split by chapter → parallel translate → combine

Reduce Strategies

- Concatenate: simple merge (summaries, translations)
- Vote/consensus: majority wins (classification tasks)
- LLM synthesis: use LLM to combine intelligently
- Hierarchical reduce: reduce in stages for large N
- Quality filter: discard low-confidence map results before reduce

Multi-Agent Comparison Matrix

Choosing the right multi-agent topology

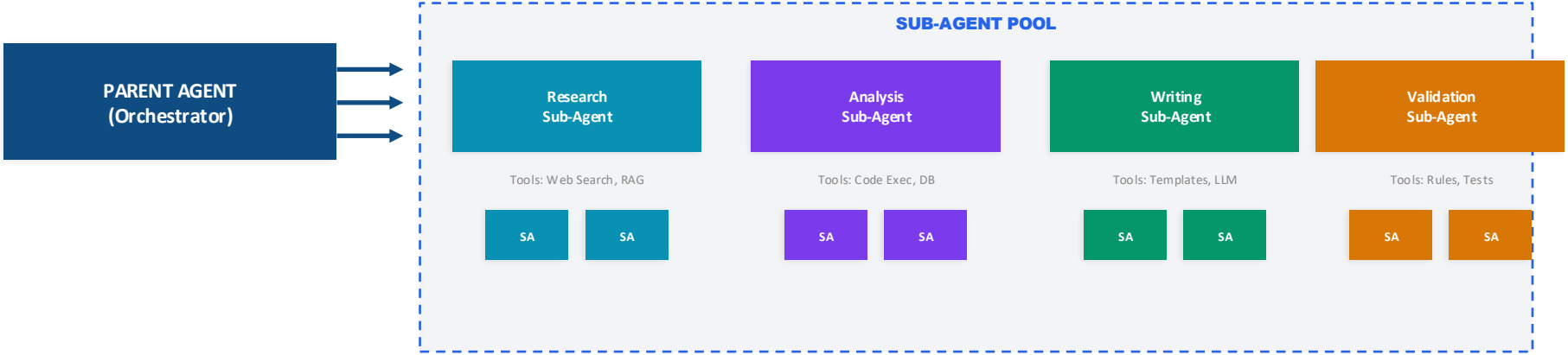
Pattern	Complexity	Fault Tolerance	Cost	Latency	Best For
Supervisor	Low	Low (SPOF)	Medium	Medium	Most enterprise use cases
Hierarchical	High	Medium	Very High	High	Large-scale decomposition
Peer-to-Peer	High	High	Very High	High	Decision quality, risk
MoA	Medium	High	High	Low-Med	Quality-critical output

Decision Framework

- Start with Supervisor — it covers 80% of enterprise multi-agent needs with lowest complexity
- Move to Hierarchical only when task decomposition has natural tree structure (>3 subtask types)
- Use Peer-to-Peer/Debate when decision quality outweighs speed and cost (risk assessment, legal, medical)
- MoA when you need ensemble quality but can afford N× cost. AaaS when building a platform for reuse

Sub-Agent Patterns — Double-Click

One agent spawning multiple sub-agents: decomposition, delegation, and lifecycle management



INPUT: Complex task + decomposition strategy + tool registry

OUTPUT: Aggregated results + execution traces + sub-agent reports

MEMORY	RUNTIME	COST	DEPTH
Parent: full | Sub: scoped context	**Parallel: O(max sub) or Sequential**	**N sub-agents × avg tokens each**	**Max 2-3 levels (avoid deep nesting)**

Sub-Agent Spawn Patterns

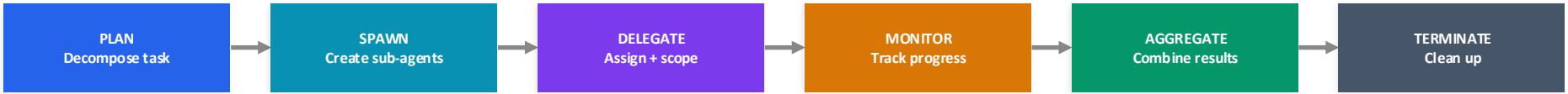
- Static: predefined sub-agents, always same team
- Dynamic: parent decides which sub-agents to spawn based on task
- Recursive: sub-agents can spawn their own sub-agents (depth limit!)

Anti-Patterns & Limits

- Nesting >3 levels deep: context loss, debugging nightmare
- No lifecycle management: zombie sub-agents consuming tokens
- No aggregation strategy: sub-agent outputs don't combine well

Sub-Agent Communication & Lifecycle

How parent agents manage sub-agent spawn, execution, and result aggregation



Sub-Agent Communication Models

Direct Report

- Sub-agent → Parent only
- Parent aggregates all results
- Simple, clean, auditable
- No cross-sub-agent comms
- Best for independent subtasks

Shared Context

- Sub-agents read/write shared store
- Results visible to siblings
- Enables collaboration
- Risk: race conditions
- Best for interdependent subtasks

Peer Messaging

- Sub-agents communicate directly
- Parent monitors, doesn't mediate
- Most flexible, most complex
- Risk: message storms, deadlocks
- Best for negotiation / debate tasks

Enterprise Best Practices for Sub-Agents

- Budget allocation: Parent sets token/time/cost budget per sub-agent. Sub-agent MUST terminate within budget — no exceptions.
- Context scoping: Give each sub-agent ONLY the context it needs. Full parent context → sub-agents is wasteful and insecure. Use context projection.
- Result schema: Define expected output format upfront. Sub-agent returns structured JSON, not free text. Parent validates schema before aggregation.
- Observability: Each sub-agent gets a child span in the trace. Parent span links all children. Log spawn/terminate events with resource metrics.

04 Swarm Patterns

Emergent intelligence — self-organizing agent collectives

Swarm Patterns

When 10+ agents self-organize — emergent behavior from simple rules

Emergent Swarm

- Agents follow simple local rules
- Global behavior emerges from interactions
- No central coordinator needed
- Inspired by ant colonies, bird flocks
- Example: web crawl swarm — each agent follows links

Memory: Distributed | Runtime: Hours+

Stigmergy Pattern

- Agents communicate through shared environment
- Leave 'pheromone trails' (metadata/signals)
- Other agents read signals to decide next action
- Self-balancing: popular paths get reinforced
- Example: task queue with priority signals

Memory: Shared env state | Runtime: Continuous

Orchestrated vs Emergent

- Orchestrated: central brain, predictable, auditable
- Emergent: no brain, adaptive, harder to debug
- Enterprise preference: orchestrated with emergent edges
- Key question: do you need auditability?
- If yes → orchestrated. If adaptability → emergent

SOLID: LSP — agents are substitutable

Swarm Scaling Patterns

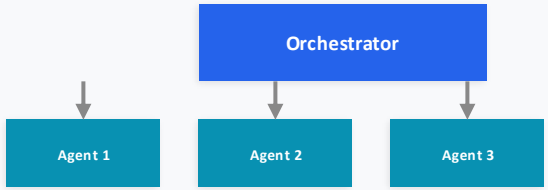
- Horizontal: add more agents of same type
- Vertical: increase agent capability/tools
- Elastic: auto-scale based on queue depth
- Self-healing: agents detect and replace failed peers
- Cost control: budget-based auto-throttling

Microservice parallel: K8s autoscaling + self-healing pods

Orchestrated vs Emergent Swarms

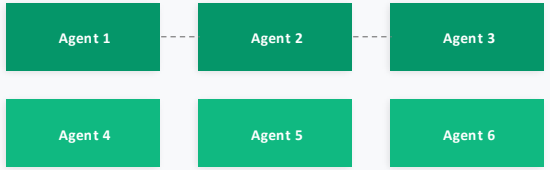
Side-by-side comparison — control versus adaptability

ORCHESTRATED SWARM



- ✓ Predictable execution order
- ✓ Full audit trail
- ✓ Deterministic cost bounds
- ✓ Easy to debug and monitor
- ✗ Single point of failure
- ✗ Less adaptive to changes

EMERGENT SWARM



- ✓ No single point of failure
- ✓ Highly adaptive and resilient
- ✓ Scales organically
- ✓ Self-healing by design
- ✗ Hard to predict behavior
- ✗ Difficult to audit/debug

Enterprise Recommendation

- Use orchestrated swarms for regulated, auditable workflows. Add emergent behavior at the edges for adaptability (hybrid approach).

05 Hybrid Patterns

Where theory meets production — 10 real-world hybrid architectures

Why Hybrid Patterns?

Real-world systems are never a single pure pattern — the 'purity gap'

TEXTBOOK

- One pattern per system
- Clean separation
- Ideal assumptions
- Academic benchmarks

≠

PRODUCTION

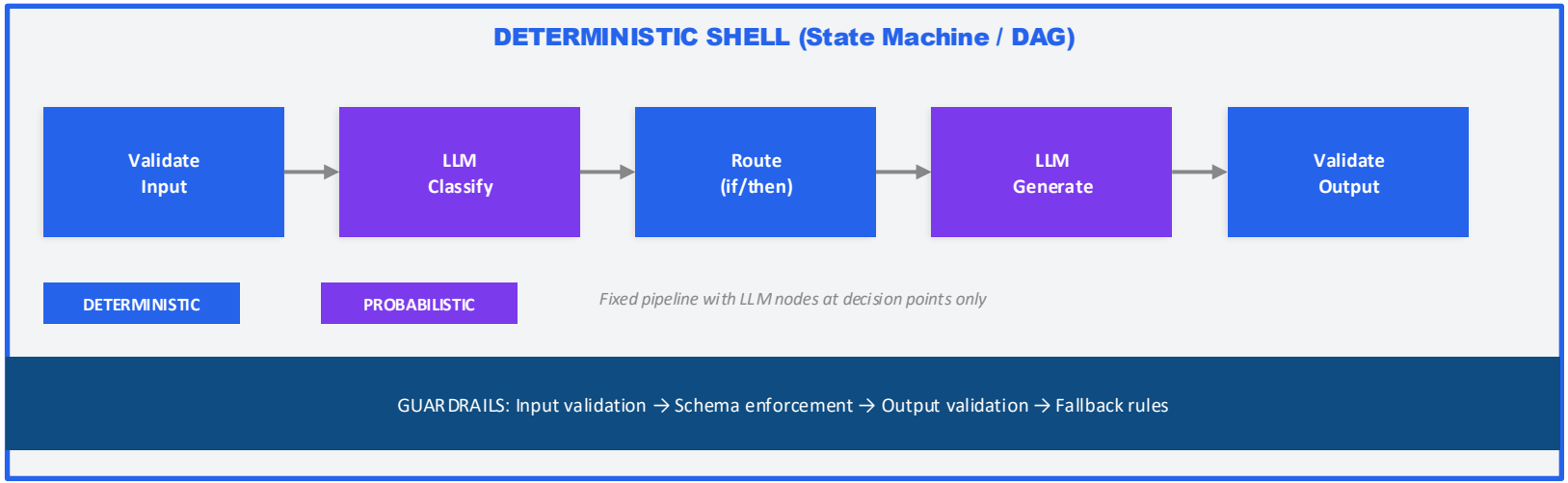
- Mixed patterns per layer
- Messy real-world constraints
- Cost/latency/compliance tradeoffs
- Hybrid by necessity

Key Hybrid Patterns Covered

Deterministic Shell + Probabilistic Core	RAG + Multi-Agent	Supervisor + Swarm	Human-in-the-Loop Hybrids	Pipeline + Agent
Router + Specialists	ReAct + RAG + Tool-Use Combo	Multi-Agent + Deep Research	Hierarchical + Peer Hybrid	Design Canvas (Build Your Own)

Hybrid 1: Deterministic Shell + Probabilistic Core

Fixed workflow wrapping LLM reasoning — guardrailed autonomy



MEMORY	RUNTIME	DETERMINISM	COST
Pipeline state (small)	**2-10s (fast, bounded)**	**High — LLM confined to slots**	**Low — minimal LLM calls**

Enterprise Use Cases

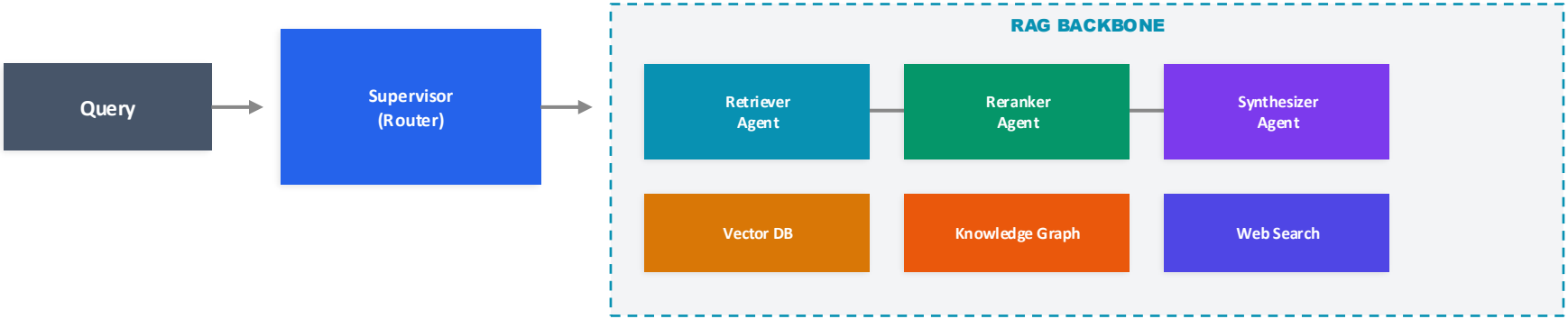
- Customer support routing with LLM classification
- Document processing pipelines with LLM extraction
- Compliance workflows with LLM risk scoring

SOLID: Open/Closed Principle

- Add new LLM nodes without changing pipeline structure
- Swap LLM providers without touching deterministic shell
- Extend validation rules without modifying core flow

Hybrid 2: RAG + Multi-Agent

Research teams with retrieval backbone — grounded multi-agent intelligence



MEMORY	RUNTIME	GROUNDING	COST
Retrieved chunks + agent context	**5-30s (retrieval + synthesis)**	**High — always cite sources**	**Medium (embedding + LLM calls)**

RAG Agent Roles

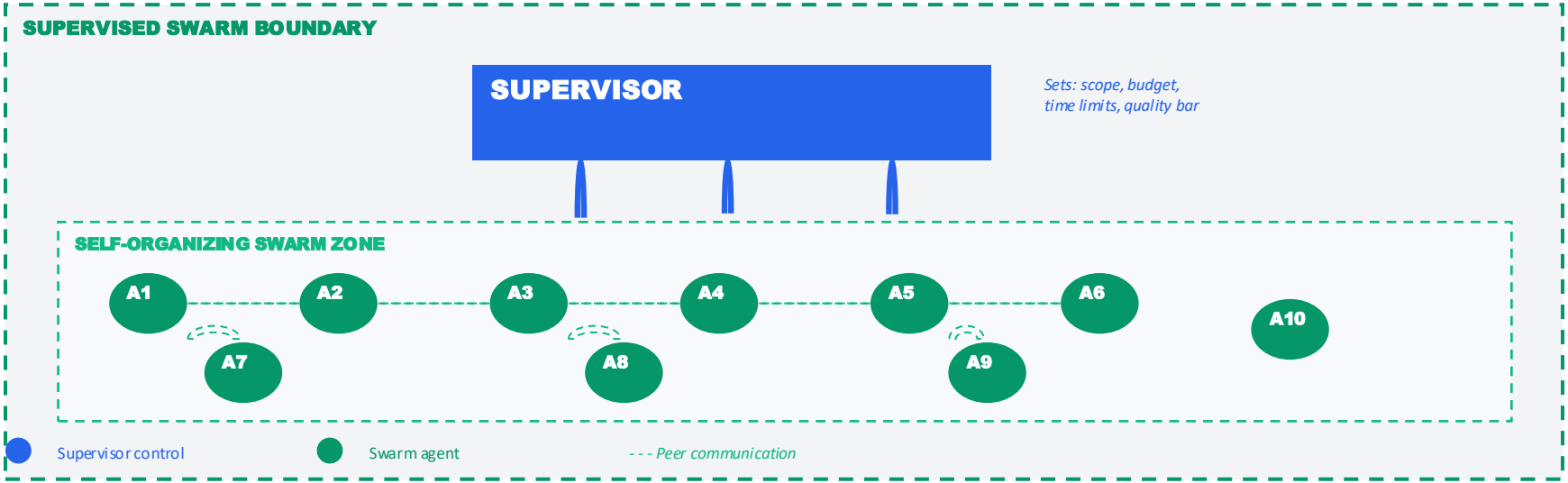
- Retriever: query expansion, multi-source retrieval
- Reranker: cross-encoder scoring, relevance filtering
- Synthesizer: combine chunks, generate grounded answer
- Validator: fact-check against retrieved sources
- Each agent = SRP (Single Responsibility Principle)

Production Patterns

- Chunking strategy: semantic > fixed-size > sentence
- Hybrid retrieval: dense vectors + sparse BM25
- Agentic RAG: agent decides when/what to retrieve
- Citation tracking: map each claim to source chunks
- Fallback: if retrieval fails, surface 'low confidence'

Hybrid 3: Supervisor + Swarm

Managed chaos — supervisor sets boundaries, swarm self-organizes within



MEMORY	RUNTIME	SCALING	RESILIENCE
Supervisor: global | Swarm: local	**Minutes to hours**	**Elastic swarm within budget**	**High — swarm self-heals**

How It Works

- Supervisor defines: task scope, budget, time limits, quality bar
- Swarm agents self-organize within these boundaries
- Supervisor intervenes only on budget/quality violations

Use Cases

- Large-scale data processing with adaptive work distribution
- Security scanning: swarm discovers, supervisor prioritizes
- Content moderation: parallel review with escalation

Hybrid 4: Human-in-the-Loop Patterns

Semi-autonomous agents with approval gates and escalation

Confidence-Based Routing

High Confidence (>90%)

Auto-approve, log for audit

Medium (70-90%)

Soft approval: notify human, auto-proceed

Low (<70%)

Hard gate: block until human approves

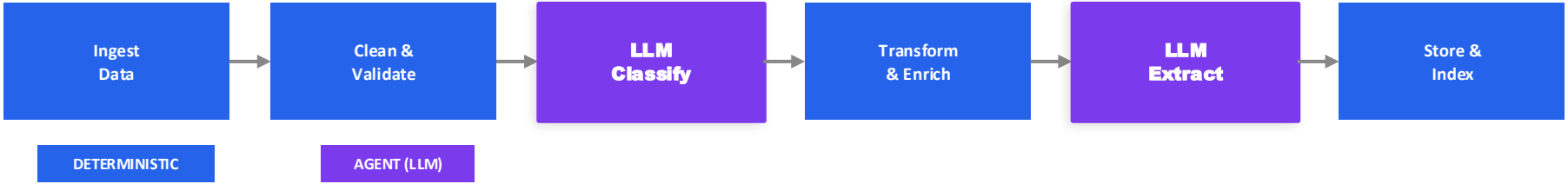
MEMORY	RUNTIME	DETERMINISM	COST
Checkpoint state at each gate	Agent: seconds | Human: minutes-hours	High at gates, variable between	Low AI cost + human time cost

HITL Design Best Practices

- Design for async: humans are slow — queue approvals, don't block. Agents should work on other tasks while waiting.
- Escalation tiers: L1 (auto) → L2 (junior human) → L3 (senior human) → L4 (committee). Match risk level to approver seniority.
- Feedback loops: human corrections feed back into agent fine-tuning. Over time, more tasks auto-approve as agent confidence improves.
- Audit everything: every gate decision (auto or human) is logged with timestamp, approver, reasoning, and confidence score.

Hybrid 5: Pipeline + Agent

Deterministic ETL stages with agent-powered decision nodes



MEMORY	RUNTIME	COST	PATTERN
Pipeline state + LLM context	**Seconds (bounded by pipeline)**	**Low — LLM only at decision nodes**	**ETL + AI = Intelligent Pipeline**

Enterprise Applications

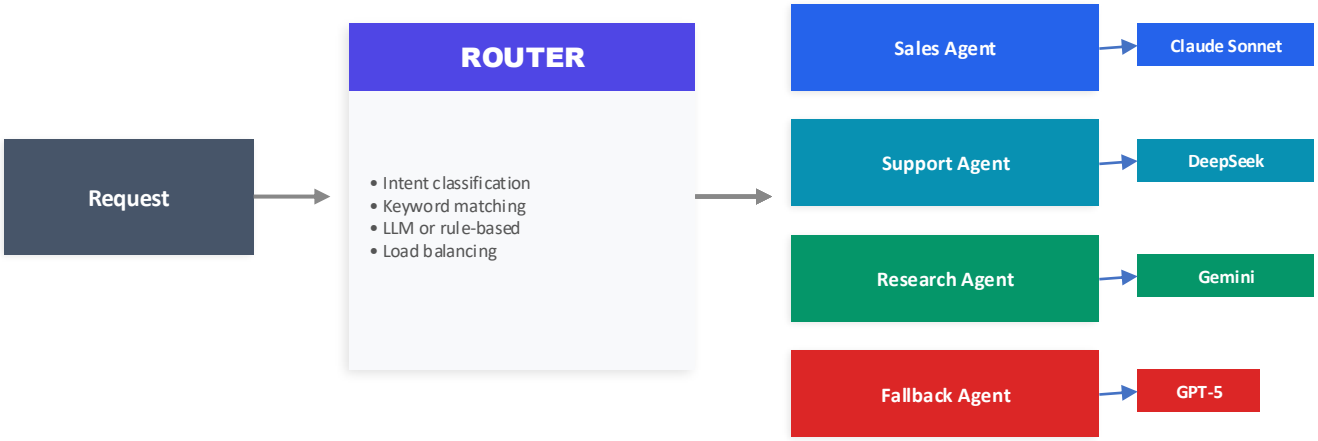
- Document processing: ingest PDF → OCR → LLM classify → extract entities → store
- Email triage: receive → parse → LLM categorize → route → archive
- Data quality: ingest → validate → LLM detect anomalies → flag → report
- Microservice parallel: middleware pipeline with AI-powered filters
- 12-Factor: each stage is a process, config-driven, log-streamed

Design Guidance

- Keep LLM nodes idempotent — same input = same output (when possible)
- Add caching layer before LLM nodes to reduce redundant calls
- Schema validation between every stage (contract testing)
- Fallback: if LLM fails, route to deterministic default or human queue
- Cost optimization: batch LLM calls, use smaller models for simple tasks

Hybrid 6: Router + Specialist Agents

API gateway pattern — deterministic routing to probabilistic specialists



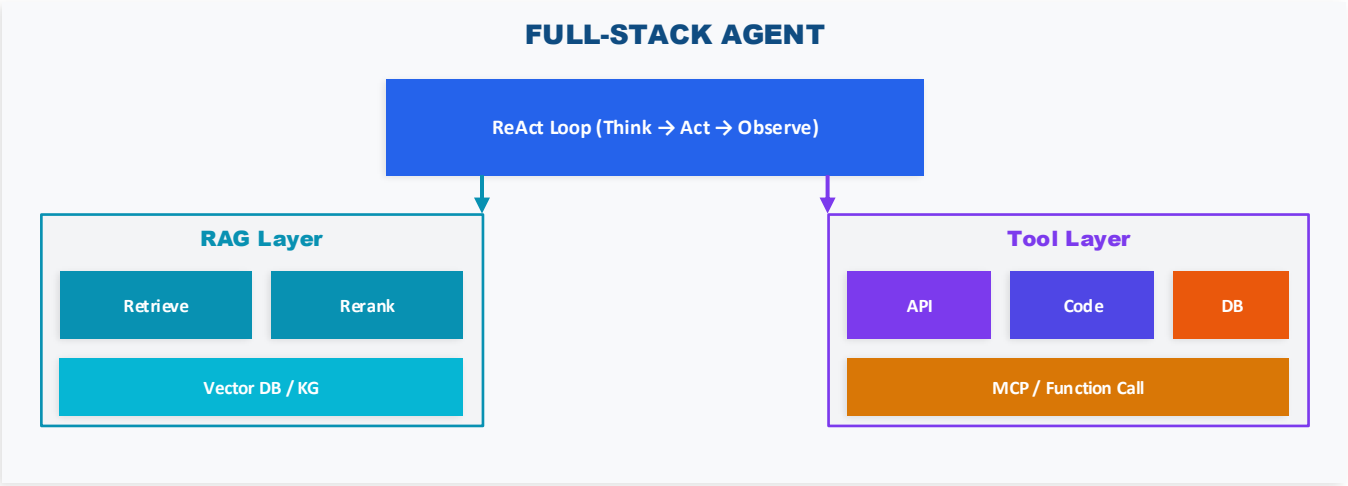
MEMORY	RUNTIME	COST	MICROSERVICE
Router: none | Agents: scoped	**Router: <1s | Agent: varies**	**Low — right-sized model per task**	**= API Gateway + Services**

Key Design Decisions

- Router type: Rule-based (faster, cheaper) vs LLM-based (more flexible). Start with rules, upgrade to LLM when rules get complex.
- Model selection per specialist: use powerful models for complex tasks, cheap models for simple ones. This is the #1 cost optimization lever.
- Fallback strategy: always have a catch-all agent. If no specialist matches, don't fail — route to a general-purpose agent or human queue.

Hybrid 7: ReAct + RAG + Tool-Use Combo

The 'full-stack agent' — the most common real-world single agent pattern



MEMORY	RUNTIME	TOKEN COST	QUALITY
Context + retrieved chunks + tool results	**5-60s depending on loops**	**High (RAG chunks + tool JSON)**	**Highest single-agent quality**

This Is Your Default Starting Point

- 90% of production agents are this hybrid. Start here, then decompose into multi-agent only when this pattern hits limits (context overflow, conflicting tools, domain specialization).
- Key optimization: smart retrieval reduces token waste. Pre-filter tools by intent classification before entering ReAct loop.

Hybrid 8: Multi-Agent + Deep Research

Orchestrated research swarms with multi-hop reasoning

MEMORY	RUNTIME	COST	QUALITY
Growing: each hop adds context	1-30 min (hop-dependent)	High — N agents × M hops	Very high — multi-source grounded

Deep Research Mechanics

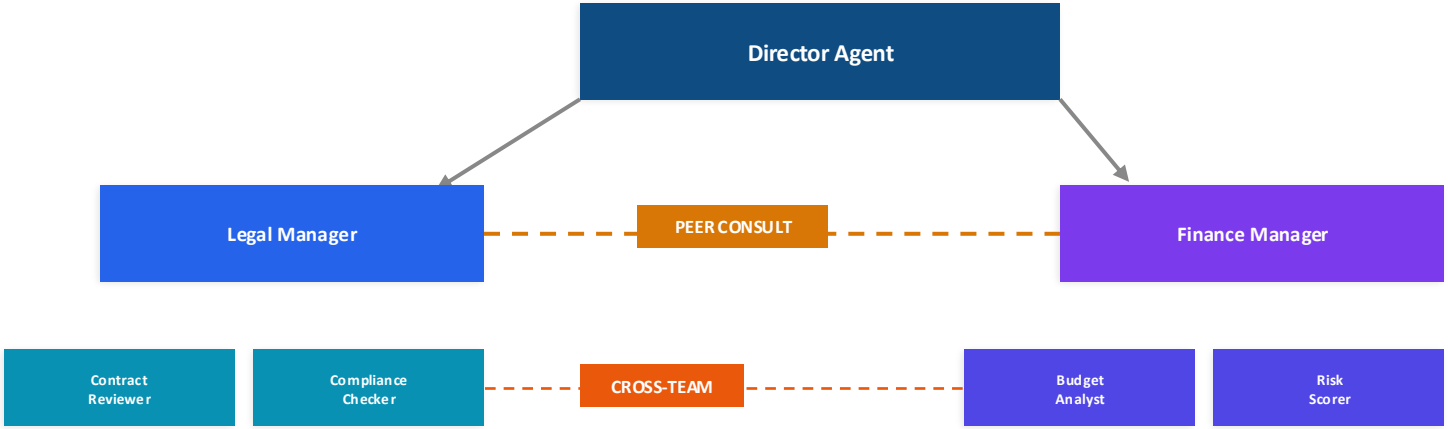
- Multi-hop: each agent's output feeds next hop's input
- Breadth-first: parallel queries across sources
- Depth-first: follow promising leads deeper
- Iterative refinement: synthesizer identifies gaps → new queries
- Citation chain: every claim traceable to source
- Deduplication: avoid redundant research across agents

Enterprise Use Cases

- Competitive intelligence: multi-source market analysis
- Due diligence: financial, legal, news, social research
- Technical research: papers, docs, code, forums
- Compliance research: regulatory changes across jurisdictions
- Patent analysis: prior art search across databases
- Memory: accumulates across hops — budget carefully!

Hybrid 9: Hierarchical + Peer Consultation

Managers delegate, but sub-agents can peer-consult before reporting back



MEMORY	RUNTIME	QUALITY	COMPLEXITY
Per-level + shared peer context	5-30 min (consultations add latency)	High — cross-functional input	High — needs clear consultation protocol

When Peer Consultation Helps

- Cross-domain decisions (legal + finance + tech)
- When sub-agents have partial information
- Risk assessment needing multiple perspectives
- Mirrors real org behavior: teams consult each other

Consultation Protocol Design

- Define who can consult whom (avoid N^2 communication)
- Set consultation time budget (don't block indefinitely)
- Structured query format: what, why, deadline
- Anti-pattern: consultation loops (A asks B asks A)

Hybrid Pattern Design Canvas

A framework for composing your own hybrid patterns

CONTROL MODEL

Deterministic / Probabilistic / Hybrid?

CARDINALITY

Single / Multi / Swarm?

COORDINATION

Supervisor / Peer / Emergent?

TOOLS & DATA

RAG / APIs / Code / Memory?

HUMAN GATES

Auto / Soft approval / Hard gate?

ERROR STRATEGY

Retry / Fallback / Escalate?

MEMORY BUDGET

Context window limit?
Shared vs isolated?

RUNTIME BUDGET

Max latency?
Async allowed?

COST BUDGET

Token cap?
Model tier per task?

How to Use This Canvas

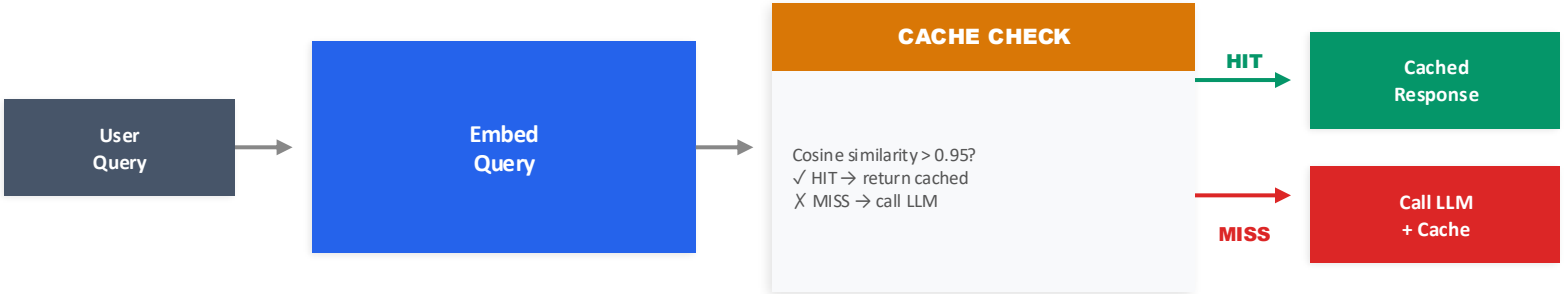
- Fill in each cell for your use case. The combination of choices defines your hybrid pattern. Most enterprise systems touch 4+ cells. No cell should be blank — even 'N/A' is a deliberate choice.

06 Tool & Capability Patterns

RAG, Deep Research, Memory, Code Execution — the agent's toolkit

Semantic Caching

Cache LLM responses for semantically similar queries — the #1 cost optimization lever



MEMORY	HIT RATE	COST SAVINGS	LATENCY
Vector index of past queries	**20-60% typical (domain-dependent)**	**20-60% LLM cost reduction**	**Cache hit: <100ms vs LLM: 1-5s**

Implementation Patterns

- Exact match: hash-based (fast, low hit rate)
- Semantic: embed + similarity search (higher hit rate)
- Hierarchical: exact → semantic → LLM fallback
- TTL-based invalidation: stale after N hours/days
- Per-agent cache: each agent type has own cache
- Shared cache: cross-agent, cross-session reuse
- Cache key: query + system prompt + tool set hash

Gotchas & Best Practices

- Similarity threshold is critical: too low = wrong answers
- Don't cache: time-sensitive, personalized, or volatile queries
- Cache warming: pre-populate with common query patterns
- Monitor hit rate + staleness metrics actively
- Security: don't serve User A's cached response to User B
- Tools: GPTCache, Redis + vector extension, Momento
- ROI: track cost-before vs cost-after per agent

Agentic RAG vs Naive RAG

The evolution from simple retrieve-generate to agent-decides-when-and-what-to-retrieve

NAIVE RAG

1. User query → embed
2. Retrieve top-K chunks (always)
3. Stuff into prompt
4. Generate answer

Problem: always retrieves, even when unnecessary. No multi-hop. No filtering. One-shot — no iteration or refinement.

AGENTIC RAG

1. Agent receives query
2. Decides: do I need to retrieve? From where?
3. Generates search queries dynamically
4. Retrieves → evaluates relevance
5. Multi-hop: follow-up queries if gaps found
6. Synthesizes grounded answer

Agent controls the retrieval loop.

NAIVE RAG COST	AGENTIC RAG COST	NAIVE RAG QUALITY	AGENTIC RAG QUALITY
Low (1 retrieval + 1 generation)	Medium-High (N retrievals + reasoning)	Low-Medium (no iteration)	High (multi-hop, verified)

When to Upgrade

- Start with Naive RAG. Upgrade to Agentic RAG when: answers are incomplete (need multi-hop), retrieval is noisy (need filtering), or questions require reasoning about WHEN and WHERE to retrieve.

Tool Design Patterns

The #1 cause of agent failure is bad tool design — get this right first

DO ✓ (Good Tool Design)

- Clear, unambiguous name: get_customer_by_id not fetch_data
- Detailed description: what it does, when to use it, edge cases
- Typed parameters with examples in schema
- Return structured data (JSON), not free text
- Idempotent operations for safe retries
- Error messages that help the LLM self-correct
- One tool = one capability (SRP from SOLID)

DON'T ✗ (Bad Tool Design)

- Vague names: process(), handle(), do_thing()
- Missing descriptions — LLM guesses incorrectly
- Untyped parameters: just 'data' with no schema
- Return raw HTML/XML that LLM can't parse
- Side effects that can't be undone on retry
- Error: 'Something went wrong' (useless to LLM)
- God tool: one function that does 10 different things

Enterprise Tool Design Checklist

- Schema: Define every tool with JSON Schema / OpenAPI spec. Include parameter descriptions, types, required fields, and example values.
- Authorization: Scope tool permissions per agent role. Research agents can read data; write agents can modify. Never give all tools to all agents.
- Rate limiting: Set per-tool rate limits. An agent calling a production API 1000x/minute will get you blocked.
- Versioning: Tools evolve. Version your tool schemas like APIs (v1, v2). Old agents can keep using old tool versions.
- Testing: Every tool needs unit tests with mocked responses AND integration tests with real APIs. Test error paths especially.
- Documentation: If the LLM can't understand the tool from its description + schema alone, it's a bad tool. Treat descriptions as critical documentation.

Tool & Capability Ecosystem

Comprehensive view — every tool pattern an enterprise agent might need

RAG

Retrieval-Augmented Generation

Mem: Chunks in context

Runtime: 2-10s

Hybridizes with: all patterns

Deep Research

Multi-hop web + doc search

Mem: Accumulates per hop

Runtime: 1-30m

Hybridizes with: all patterns

Code Execution

Run code in sandbox

Mem: Execution state

Runtime: 1-30s

Hybridizes with: all patterns

Function Calling

Structured API invocation

Mem: Stateless

Runtime: <5s

Hybridizes with: all patterns

Memory

Short/Long-term retrieval

Mem: Persistent store

Runtime: 0.1-2s

Hybridizes with: all patterns

Web Browsing

Navigate, extract, interact

Mem: Page context

Runtime: 5-60s

Hybridizes with: all patterns

RAG Pattern — Double-Click

Retrieval-Augmented Generation: grounding LLM output in real data

MEMORY	RUNTIME	COST	QUALITY
Retrieved chunks fill context window	**2-10s (embed + search + generate)**	**Embedding cost + LLM generation**	**Grounded — reduces hallucination**

Chunking & Retrieval Strategies

- Semantic chunking: split by meaning boundaries
- Fixed-size with overlap: simple but effective (512 tokens, 50 overlap)
- Hierarchical: parent-child chunk relationships
- Hybrid retrieval: dense vectors (semantic) + BM25 (keyword)
- Multi-index: different indices for different doc types
- Agentic RAG: agent decides what/when to retrieve
- Query expansion: rephrase query for better recall
- HyDE: generate hypothetical answer, embed, search
- Memory impact: each chunk = ~100-500 tokens in context

Enterprise RAG Architecture

- Ingestion pipeline: source → clean → chunk → embed → index
- Vector DB: Pinecone, Weaviate, pgvector, Qdrant
- Reranking: cross-encoder model for precision
- Citation: map each generated claim to source chunk
- Access control: filter chunks by user permissions
- Freshness: incremental indexing, TTL on chunks
- Evaluation: answer relevance, faithfulness, context recall
- Cost: embedding is cheap; reranking adds ~2s; worth it
- Anti-pattern: stuffing too many chunks → noise drowns signal

Memory Patterns — Double-Click

How agents remember — from conversation buffers to persistent knowledge graphs

Working Memory	Current context window. Limited by model's token limit. Grows per interaction.	Grows linearly with tokens Single session
Short-Term Memory	Summarized conversation history. Compressed context for multi-turn interactions.	Summary generation cost Conversation
Episodic Memory	Past task experiences. 'I tried X and it failed, so try Y.' Learning from history.	Storage + retrieval Cross-session
Semantic Memory	Facts & knowledge in vector DB or knowledge graph. External long-term storage.	Embedding + vector DB Persistent
Shared Memory	Blackboard / shared state between agents. Enables coordination without direct comms.	Sync + conflict resolution Multi-agent session

07 Orchestration & Communication

How agents coordinate — routers, DAGs, protocols, and error handling

Orchestration Patterns

Five ways to coordinate agent workflows

Router

Single decision point dispatches to the right agent/path. Fast, simple.

Use: Intent classification, request routing

MEM: Minimal

ROUTER

Pipeline

Sequential stages, each transforms data for the next. Predictable flow.

Use: Data processing, document workflows

MEM: Per-stage state

PIPELINE

DAG

Directed Acyclic Graph — parallel branches with dependencies. Maximum parallelism.

Use: Complex workflows with independent subtasks

MEM: Per-node + dependency state

DAG

State Machine

Finite states with defined transitions. Each state = agent behavior. Highly auditable.

Use: Approval flows, order processing, compliance

MEM: State + transition log

STATE MACHINE

Blackboard

Shared workspace all agents read/write. No direct communication between agents.

Use: Collaborative problem-solving, research

MEM: Shared store (growing)

BLACKBOARD

Communication Patterns

How agents talk to each other — four fundamental approaches

Message Passing

Direct agent-to-agent messages. Simple, explicit, easy to trace.

REST / gRPC calls

✓ Clear, debuggable, ordered

✗ Tight coupling, N^2 connections

Pub/Sub (Event-Driven)

Agents publish events, others subscribe to topics of interest.

Kafka / RabbitMQ

✓ Loose coupling, scalable

✗ Harder to trace, eventual consistency

Shared Memory

Agents read/write to a shared store (blackboard, database, cache).

Redis / DB shared state

✓ Simple coordination, no messaging infra

✗ Race conditions, lock contention

Streaming

Continuous token-level output shared in real-time between agents.

SSE / WebSocket / gRPC stream

✓ Low latency, progressive output

✗ Complex error handling, backpressure

Context Window Management

The hidden bottleneck — strategies for managing the most expensive resource: tokens

Sliding Window	Keep last N messages, drop oldest. Simple but loses early context.	Cost: Fixed Quality: Medium
Summarization	Periodically summarize conversation history into a compact form.	Cost: Summary gen cost Quality: High
RAG Augmented	Store history in vector DB, retrieve only relevant past context.	Cost: Embed + retrieve Quality: High
Hierarchical	Full recent + summarized mid + key facts from distant past.	Cost: Medium Quality: Highest
Token Budgeting	Allocate fixed token budgets per component: system, tools, history, response.	Cost: N/A (planning) Quality: Controlled

Rule of Thumb

- Budget: system prompt (10-15%) + tools (20-30%) + history (30-40%) + response (20-30%). Never let any one component exceed 50%.

Protocols & Standards

MCP, A2A, and the emerging agent communication landscape

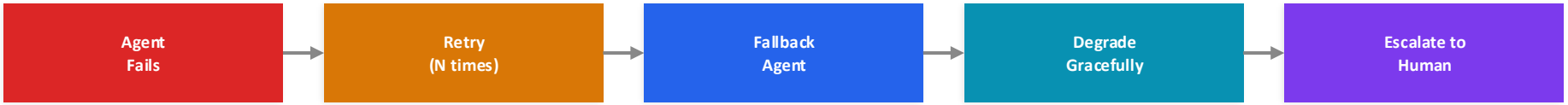
MCP Model Context Protocol	By: Anthropic	Tool discovery & invocation	Production
A2A Agent-to-Agent Protocol	By: Google	Agent interoperability	Emerging
OpenAI Tools Function Calling Spec	By: OpenAI	Structured tool use	Production
LangGraph Graph Orchestration	By: LangChain	Stateful multi-agent flows	Production
AutoGen Multi-Agent Framework	By: Microsoft	Conversational multi-agent	Production

Recommendation

- Use MCP for tool integration, LangGraph/AutoGen for orchestration, and watch A2A for cross-vendor agent interop.

Error Handling, Retry & Fallback Chains

Design for failure — because agents will fail



RETRY STRATEGY
Exponential backoff + jitter

CIRCUIT BREAKER
Open after N failures, half-open test

FALLBACK CHAIN
Primary → Secondary → Degraded → Human

TIMEOUT
Per-agent + per-workflow limits

Failure Modes & Mitigations

- LLM timeout: set hard timeout, retry with shorter prompt
- Hallucination: validation layer, confidence scoring
- Tool failure: retry with backoff, fallback to alternative tool
- Context overflow: summarize & compress, or split task
- Rate limit: queue management, request batching
- Cascade failure: circuit breaker per agent
- Zombie agent: watchdog timer, force termination
- Data corruption: idempotent operations, rollback capability
- 12-Factor: treat logs as event streams for debugging

Enterprise Error Patterns

- Dead Letter Queue: failed tasks stored for analysis
- Compensating Transaction: undo completed steps on failure
- Bulkhead: isolate agent pools to prevent cascade
- Chaos engineering: intentionally inject failures to test resilience
- Correlation IDs: trace errors across agent chains
- Alert thresholds: auto-alert on error rate spikes
- Runbook automation: known errors trigger automated fixes
- Post-mortem: every incident → updated error handling
- Cost of failure: budget for retries in token cost estimates

08 Core Engineering Principles

SOLID, Microservices, 12-Factor — proven principles applied to agents

SOLID Principles Applied to Agents

Time-tested software principles mapped to agent architecture

S	**Single Responsibility**	One agent = one capability. Research agent only researches. Don't build 'god agents'.
O	**Open/Closed**	Extend agent behavior via new tools/prompts, not by modifying core agent code.
L	**Liskov Substitution**	Any agent implementing an interface can be swapped. GPT ↔ Claude ↔ Gemini for same task.
I	**Interface Segregation**	Agents expose minimal contracts. Supervisor doesn't need to know worker internals.
D	**Dependency Inversion**	Depend on abstractions (tool interface) not implementations (specific API). Use MCP.

Microservices ↔ Agent Architecture Mapping

Every microservice pattern has an agent equivalent

Microservice Pattern	Agent Equivalent	What It Does
API Gateway	Router Agent	Routes requests to the right service/agent
Service Mesh	Agent Mesh (AaaS)	Service discovery, load balancing, observability
Saga Pattern	Multi-Agent Orchestration	Distributed transaction across services/agents
Circuit Breaker	Agent Fallback Chain	Detect failure, stop cascading, use fallback
Sidecar/Ambassador	Tool Proxy Layer	Add capabilities without changing core agent
Event Sourcing	Agent Trace Log	Store every event/decision for replay & audit
CQRS	Read Agent / Write Agent	Separate query agents from action agents
Bulkhead	Agent Pool Isolation	Isolate agent groups to prevent cascade failure

12-Factor & Cloud-Native Principles for Agents

Building agents that are production-ready from day one

I — Codebase

One repo per agent type, tracked in version control

II — Dependencies

Declare model, tools, and SDK versions explicitly

III — Config

Prompts, model params, tool configs in env vars — not code

IV — Backing Services

LLMs, vector DBs, tools are attached resources — swappable

V — Build/Release/Run

Separate prompt engineering from deployment from runtime

VI — Processes

Agents are stateless processes — state in external stores

VII — Port Binding

Agents self-contain as services, expose via API

VIII — Concurrency

Scale by running more agent instances, not bigger agents

IX — Disposability

Fast startup, graceful shutdown. Agent can be killed anytime.

X — Dev/Prod Parity

Same prompts, same tools in dev and prod. Mock LLMs for tests.

XI — Logs

Stream all agent decisions, tool calls, results as event logs

XII — Admin Processes

One-off tasks (migration, cleanup) run as separate agents

Prompt Management Patterns

Version, test, optimize, and deploy prompts like production code

Prompt Versioning

- Git-like version control for all prompts
- Semantic versioning: v1.0.0 → v1.1.0
- Diff tracking: what changed between versions
- Rollback capability on regression
- Environment promotion: dev → staging → prod
- Immutable: never edit in-place, always new version
- Tools: PromptLayer, Humanloop, LangSmith

Prompt Testing

- Unit tests: expected output for known inputs
- Regression tests: ensure old behaviors preserved
- A/B testing: compare prompt versions on live traffic
- Red-teaming: adversarial inputs to find failure modes
- Eval sets: curated test cases per agent capability
- Assertion-based: check output structure, not just content
- Tools: Promptfoo, DeepEval, RAGAS

Prompt Optimization

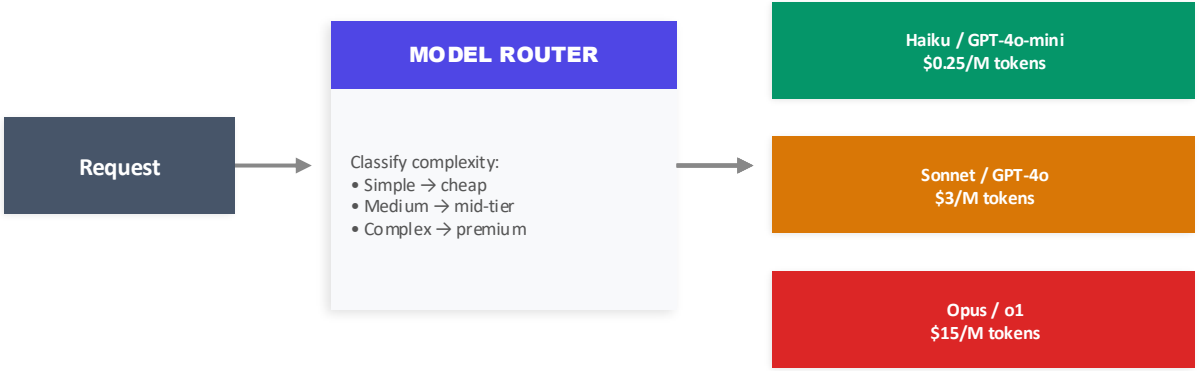
- DSPy: programmatic prompt optimization
- Few-shot selection: auto-select best examples
- Chain-of-thought: auto-generate reasoning steps
- Meta-prompting: use LLM to improve prompts
- Compression: reduce token count without quality loss
- Temperature tuning per task type
- Model-specific: same prompt ≠ same results across models

Enterprise Prompt Lifecycle

- Write → Test (eval set) → Review (peer + automated) → Stage (shadow traffic) → Deploy (canary rollout) → Monitor (quality + cost metrics) → Iterate
- Treat prompts as first-class artifacts: reviewed, versioned, tested, deployed through CI/CD. Never 'just paste it in'. Prompt drift is the silent killer of agent quality.
- Separation of concerns: prompt engineers write prompts, infra team deploys them, monitoring team tracks quality. Don't conflate these roles.

Model Routing & Selection

Right-size your models — use expensive models for hard tasks, cheap for easy



Typical savings:
40-70% cost reduction
with <5% quality loss

ROUTER TYPE	ROUTING LATENCY	COST IMPACT	QUALITY IMPACT
Rule / Classifier / LLM-based	**<100ms (classifier) to 1s (LLM)**	**40-70% savings on blended cost**	**<5% degradation if well-tuned**

Routing Strategies

- Complexity classifier: train on labeled difficulty data
- Keyword heuristics: math/code → premium, FAQ → cheap
- Cascading: try cheap first, escalate on low confidence
- Task-type routing: summarization=cheap, analysis=premium
- User tier: enterprise=premium, free=cheap

Implementation Tips

- Start with simple rules, upgrade to classifier when data exists
- Monitor quality per tier: are cheap-model answers acceptable?
- Track cost-per-quality: $/quality-score ratio per model
- A/B test routing decisions to optimize thresholds
- Tools: Martian, Unify, OpenRouter, custom gateway

Design for Failure

Circuit breakers, graceful degradation, and idempotency in agent systems

Circuit Breaker

- Monitor agent failure rate
- CLOSED: normal operation
- OPEN: too many failures, skip agent, use fallback
- HALF-OPEN: test with single request, decide to close or stay open
- Runtime: adds ~10ms check overhead

Graceful Degradation

- Level 1: Full agent response
- Level 2: Cached/pre-computed response
- Level 3: Simplified rule-based fallback
- Level 4: Static default response
- Level 5: Honest 'service unavailable' with ETA

Idempotency

- Same input → same output (when possible)
- Use idempotency keys for tool calls
- Critical for retries: won't double-execute
- Store execution receipts for deduplication
- Challenge: LLMs are non-deterministic by nature

Resilience Checklist

- ☐ Every agent has a timeout (hard limit)
- ☐ Every external call has retry + backoff
- ☐ Circuit breakers on all inter-agent calls
- ☐ Fallback chain defined for critical paths
- ☐ Correlation ID flows through entire chain
- ☐ Dead letter queue for failed tasks

Cost of Failure Budget

- Factor retries into token cost estimates: 1.3-1.5× base
- Budget for fallback agent calls: cheaper models as backup
- Human escalation has hidden cost: time + context switching
- Monitor: cost per successful completion, not per attempt
- Set alerts: cost > 2× expected → investigate, not auto-retry
- Memory: retries accumulate context → compress between retries

Anti-Patterns to Avoid

Common mistakes in enterprise agent design — DO vs DON'T

DO ✓

- Single Responsibility — one agent, one job
- Schema-first tool design with validation
- Set hard token/time/cost budgets per agent
- Implement observability from day one
- Test with adversarial inputs before production
- Use structured output (JSON) for inter-agent comms
- Version your prompts like you version code
- Start simple (single agent), scale to multi only when needed
- Design for async — humans and external services are slow
- Log everything: decisions, tool calls, results, errors

DON'T ✗

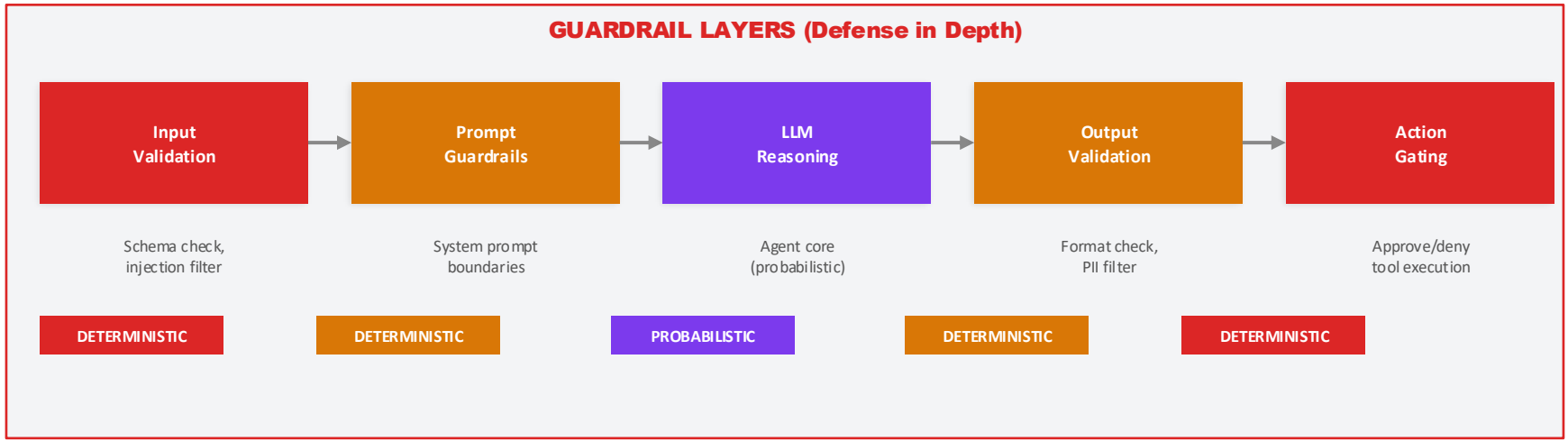
- God Agent — one agent that does everything
- Chatty Agents — excessive inter-agent messages
- No Observability — can't debug what you can't see
- Tight Coupling — agents depend on other agents' internals
- Infinite Loops — no max iteration or timeout limits
- Prompt-in-code — hardcoded prompts that can't be updated
- Ignore costs — LLM calls are not free, budget them
- Trust blindly — validate LLM output before acting on it
- Over-engineer early — don't build swarms for simple tasks
- Skip testing — 'it works in the playground' ≠ production-ready

09 Enterprise Governance & Selection

Guardrails, observability, cost control, and the decision matrix

Guardrails, Approval Gates & Safety Layers

Enterprise-grade controls for agent autonomy



Safety Controls

- PII detection + redaction before LLM processing
- Content moderation on inputs and outputs
- Tool allowlist: agents can only use approved tools
- Budget limits: hard cap on tokens / API calls / cost
- Rate limiting: prevent agent from spamming external APIs

Compliance & Audit

- Every decision logged with reasoning trace
- Immutable audit log for regulatory compliance
- Data residency: control where LLM calls are processed
- Model governance: approved model list per task type
- Prompt versioning: track changes like code deployments

Observability, Tracing & Cost Monitoring

You can't manage what you can't measure — full visibility into agent operations

Metrics

- Tokens per request (input + output)
- Latency per agent step
- Tool call success/failure rate
- Task completion rate
- Cost per completion
- Memory usage over time
- Agent retry rate
- Queue depth (for async agents)

Tracing

- Distributed trace per request
- Span per agent in multi-agent flows
- Parent-child spans for hierarchical delegation
- Tool call traces with request/response
- LLM reasoning captured per step
- Correlation IDs across agent boundaries
- OpenTelemetry compatible
- Tools: LangSmith, Arize, Langfuse

Logging

- Structured JSON logs (not free text)
- Decision logs: what agent decided and why
- Tool invocation logs with payloads
- Error logs with stack trace + context
- Prompt versions logged per execution
- User feedback logged for fine-tuning
- PII scrubbed before logging
- Log retention policy per compliance needs

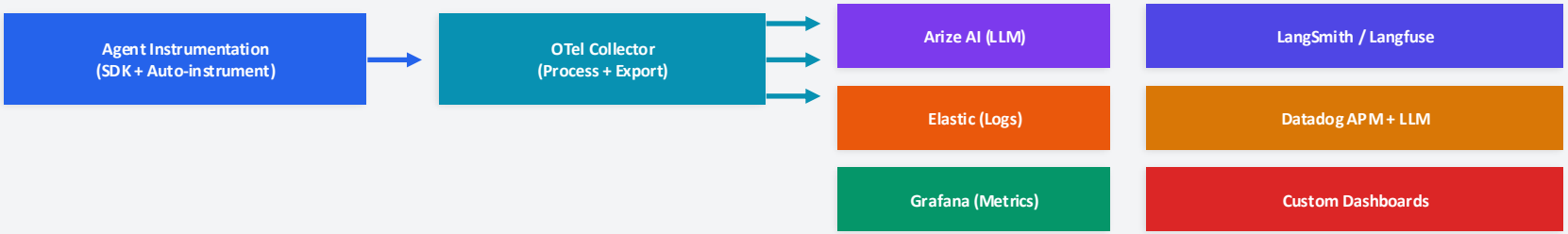
Cost Monitoring Dashboard (Key Metrics)

- Total spend: daily / weekly / monthly breakdown by model, agent, and team
- Cost per task: track completion cost, not just API call cost. Include retries, fallback calls, and human escalation time.
- Budget alerts: 50% → yellow, 80% → orange, 95% → red auto-throttle. Per-agent and per-workflow budgets.
- ROI tracking: cost of agent vs cost of human doing same task. Justify agent investments with data.
- Optimization leaderboard: which agents are most/least cost-efficient? Target optimization efforts.

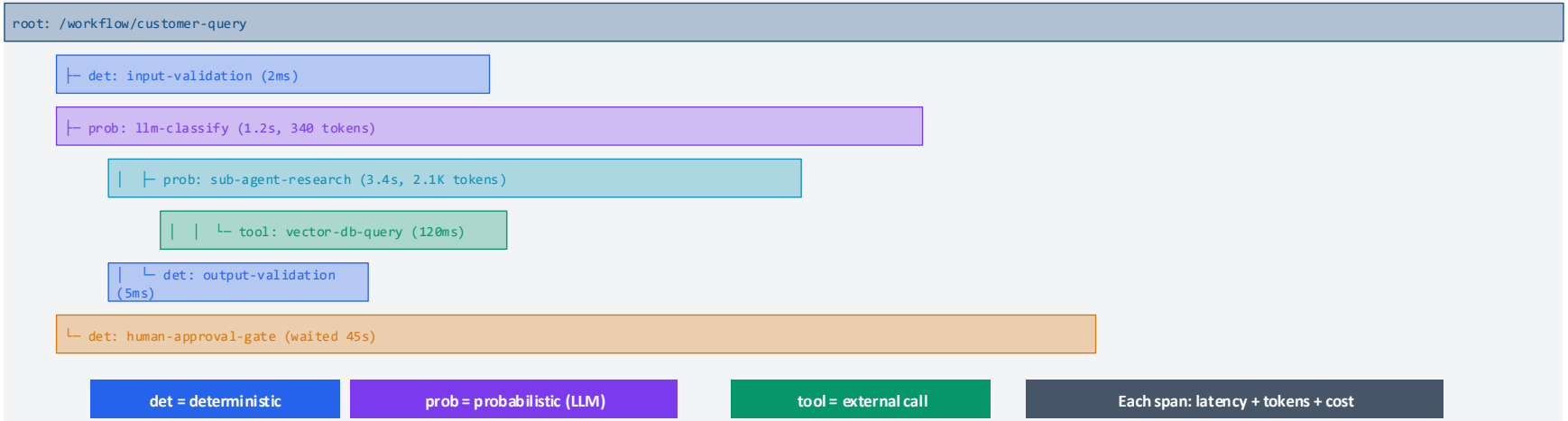
Observability for Complex Hybrid Systems

OpenTelemetry, Arize AI, Elastic — full-stack tracing for multi-agent + hybrid architectures

OBSERVABILITY ARCHITECTURE FOR HYBRID AGENT SYSTEMS



Distributed Trace Structure for Hybrid Agent Flow



Observability Tooling & Best Practices

OpenTelemetry as the backbone — Arize AI for LLM-specific, Elastic for logs, Grafana for metrics

OpenTelemetry (OTel)

- Vendor-neutral standard for traces, metrics, logs
- Auto-instrument LLM SDKs (Anthropic, OpenAI)
- Custom spans for agent reasoning steps
- Semantic conventions for GenAI (new spec)
- Export to any backend: Elastic, Datadog, Grafana
- Agent-specific attributes: model, tokens, cost
- Propagate trace context across agent boundaries

Arize AI

- Purpose-built for LLM/AI observability
- Prompt + response logging with embeddings
- Hallucination detection (similarity scoring)
- Drift monitoring: quality degradation alerts
- Eval dashboards: score distributions over time
- A/B testing for prompt versions
- Cost attribution per model, agent, workflow

Elastic Stack

- ELK: Elasticsearch + Logstash + Kibana
- Structured agent logs → Elasticsearch index
- Kibana dashboards: real-time agent activity
- APM integration: distributed tracing in Kibana
- ML anomaly detection on agent behavior
- Long-term log retention for compliance audits
- Full-text search across all agent conversations

Hybrid System Observability Checklist

- Trace structure: Root span (workflow) → child spans (deterministic stages) → nested spans (LLM calls, tool calls, sub-agents). Tag each span with control_type=det/prob.
- Token accounting: Every LLM span must log: model, input_tokens, output_tokens, cost, latency. Aggregate at workflow level for total cost per request.
- Agent state logging: Log agent decisions (which tool, why), not just tool call results. This is your debug lifeline when things go wrong.
- Cross-boundary tracing: When Agent A delegates to Agent B (or sub-agents), propagate trace context. Use W3C Trace Context standard via OTel.
- Alerting tiers: P1 (agent failure rate >5%) → page. P2 (cost per request >2× baseline) → Slack. P3 (latency P95 >30s) → email. P4 (quality drift) → daily report.

Memory & Runtime Architecture

How memory and compute scale across pattern types — the cost equation

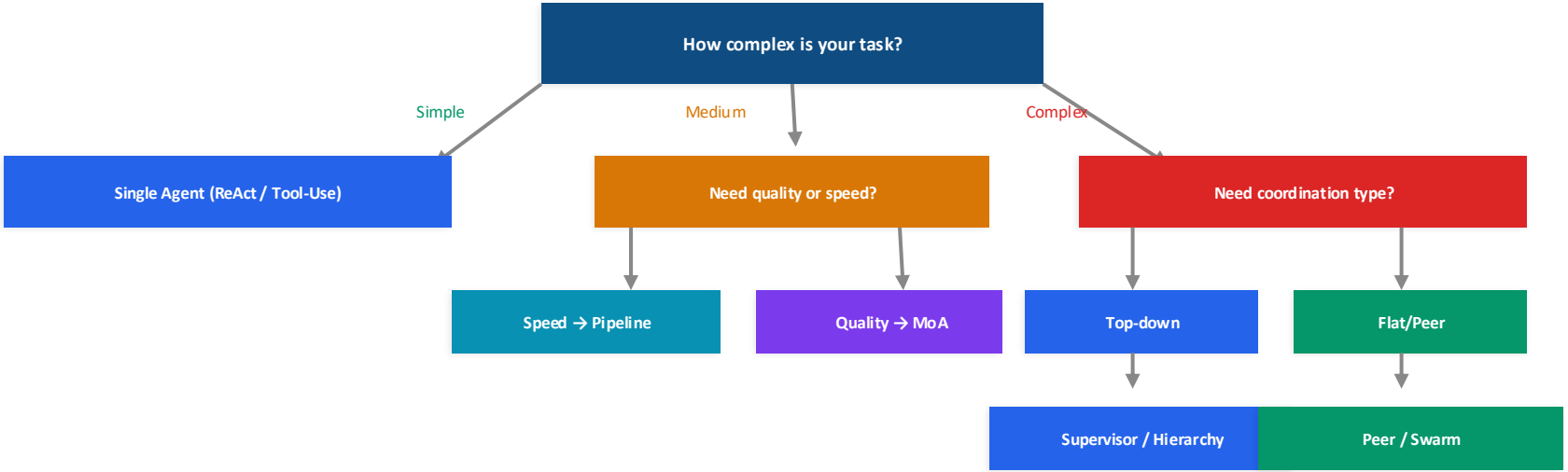
Pattern	Memory Model	Token Budget	Runtime Profile	Cost Scaling
Single (ReAct)	Context window grows per loop	2K-32K tokens	Linear: O(loops)	$$
Single (Tool-Use)	Stateless per call	1K-8K tokens	Constant: O(1)	$
Multi (Supervisor)	Supervisor: full / Workers: scoped	32K-128K total	Sequential: O(workers)	$$$
Multi (Peer/Debate)	Shared + individual per agent	N × 32K tokens	Rounds: O(agents × rounds)	$$$$
Swarm	Distributed, local per agent	N × 8K tokens	Parallel: O(max agent)	$$$$$
Hybrid (Shell+Core)	Pipeline state + LLM context	2K-16K tokens	Bounded: O(pipeline stages)	$$

Key Insight

- Memory is your #1 cost driver. Token budgets compound in multi-agent systems. Always calculate total token cost = Σ(agents × avg_tokens × iterations) before choosing a pattern.

Pattern Selection Decision Matrix

Flowchart — pick the right pattern for your use case



Universal Add-Ons (apply to any pattern):

+RAG | +Memory | +HITL Gates | +Guardrails | +Observability | +Cost Controls

The Golden Rule

- Start with the simplest pattern that solves your problem (usually Single Agent + Tool-Use). Add complexity only when you hit measurable limits: context overflow, conflicting tools, latency requirements, or domain specialization needs. Every added agent multiplies cost, latency, and debugging complexity.

Security Patterns for Agent Systems

Prompt injection, sandboxing, and adversarial defense — the attack surface is new and large

Attack Vectors

- Direct prompt injection: user manipulates system prompt
- Indirect injection: malicious content in retrieved docs/tools
- Tool abuse: agent tricked into calling dangerous tools
- Data exfiltration: agent leaks sensitive context to external API
- Privilege escalation: agent bypasses intended access scope
- Denial of service: infinite loops or massive token generation
- Supply chain: compromised tool/MCP server

Defense Patterns

- Input sanitization: strip known injection patterns
- Output validation: check for PII, secrets, policy violations
- Least privilege: agents get only the tools they need
- Sandboxing: code execution in Docker/Firecracker
- Network isolation: agents can't access arbitrary URLs
- Content boundary: separate system/user/tool content clearly
- Canary tokens: detect if agent context is leaked

Enterprise Security Architecture

- Layer 1 — Input Gate: sanitize + classify input. Block obvious injection attempts. Rate limit per user/session.
- Layer 2 — Execution Sandbox: all tool calls run in isolated environments. Network egress whitelisted. File system restricted. CPU/memory limits enforced.
- Layer 3 — Output Gate: PII detection + redaction. Policy compliance check. Content safety filter. Secrets scanning (no API keys in responses).
- Layer 4 — Monitoring: anomaly detection on agent behavior. Alert on unusual tool call patterns, unexpected data access, or cost spikes. Red-team regularly.
- Layer 5 — Audit: immutable log of all inputs, reasoning, tool calls, outputs. Correlation IDs for cross-agent tracing. Retention per compliance requirements.

Evaluation & Testing Patterns

How to test agents — because 'it works in the playground' is not a test strategy

Unit Testing

- Test individual tools in isolation
- Mock LLM responses for deterministic tests
- Validate tool schema + response formats
- Test error handling paths explicitly
- Test prompt variations for robustness
- Run fast: <1 min for full suite
- Tools: pytest, Jest, mock frameworks

Integration Testing

- Test full agent pipeline end-to-end
- Use real LLM calls (budget for test runs)
- Golden dataset: known input → expected output
- Multi-turn conversation testing
- Tool chain testing: sequences of tool calls
- Run daily: 10-30 min for full suite
- Tools: LangSmith, Promptfoo, custom harness

Evaluation Metrics

- Task completion rate (binary pass/fail)
- Answer quality (LLM-as-judge scoring)
- Faithfulness: are claims grounded in sources?
- Hallucination rate: false info in output
- Tool accuracy: right tool, right parameters
- Latency: time to completion (P50/P95/P99)
- Cost per task: token spend per completion

Red-Teaming & Adversarial Testing

- Prompt injection attempts across all input vectors
- Edge cases: empty input, max length, unicode, multi-language
- Role confusion: trick agent into ignoring instructions
- Tool abuse: manipulate agent into unintended tool use
- Data leakage: attempt to extract system prompt / context
- Automated red-teaming: use LLMs to generate attacks

Continuous Evaluation

- Shadow testing: new version runs alongside prod, compare results
- Canary deployment: 5% traffic to new agent, monitor quality
- Online evaluation: sample prod conversations, score async
- Drift detection: quality score trending down? Alert!
- User feedback loop: thumbs up/down → label data → improve
- Regression suite: run on every prompt/tool change before deploy

Governing Hybrid Systems

Tracing, debugging, and cost attribution across mixed deterministic/probabilistic flows

Trace Architecture for Hybrids

- Root span: entire workflow request
- Child spans: each deterministic stage
- Nested spans: LLM calls within stages
- Tags: deterministic=true/false per span
- Metrics: token count, latency, cost per span
- Correlation: link human approval events to trace

Debugging Hybrid Failures

- Challenge: was it the deterministic or probabilistic part?
- Solution: tag every span with control_type=det/prob
- Replay: deterministic parts are reproducible
- LLM parts: log full prompt + response for replay
- Root cause: start from output, trace backwards
- Diff tool: compare successful vs failed traces

Cost Attribution

- Per-component: attribute costs to det vs prob layers
- Per-team: which team's agents are most expensive?
- Per-task: what does each workflow type cost?
- Chargeback: allocate AI spend to business units
- Showback: visibility without actual billing
- Optimization: identify highest-cost components first
- Trend analysis: cost growth rate per pattern type

Governance Checklist

- ☐ All agent patterns registered in service catalog
- ☐ Approved model list per task classification
- ☐ Token budgets set and enforced per workflow
- ☐ Observability dashboards per team
- ☐ Incident response playbook for agent failures
- ☐ Monthly cost review with optimization targets
- ☐ Quarterly pattern review: right pattern for each task?

Agent Framework Comparison

LangGraph vs CrewAI vs AutoGen vs Semantic Kernel — choosing your foundation

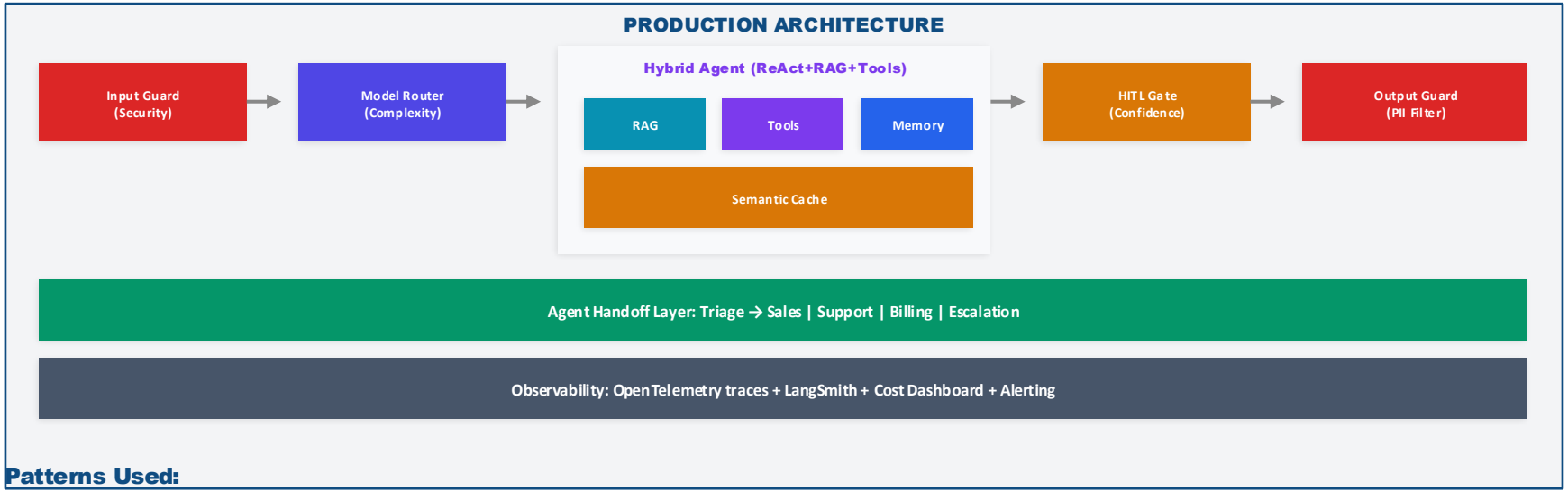
Framework	By	Pattern	Strength	Weakness	Best For
LangGraph	LangChain	Graph / State Machine	Flexible, production-ready, great debugging	Steeper learning curve	Complex stateful workflows
CrewAI	CrewAI	Role-based multi-agent	Easy to start, intuitive role model	Less flexible orchestration	Quick multi-agent prototypes
AutoGen	Microsoft	Conversational multi-agent	Rich conversation patterns, CodeAct native	Complex setup for simple tasks	Research, code gen, analysis
Semantic Kernel	Microsoft	Plugin-based agents	Enterprise .NET/Java, strong typing	Smaller community, enterprise focus	.NET/Java enterprise shops
DSPy	Stanford	Programmatic optimization	Auto-optimize prompts + pipelines	Not an agent framework per se	Prompt optimization layer
OpenAI Swarm	OpenAI	Agent handoff	Dead simple, minimal abstraction	Limited orchestration patterns	Simple handoff workflows

Recommendation

- LangGraph for production-grade enterprise. CrewAI for fast prototyping. AutoGen for research/code. DSPy as optimization layer on top of any framework.

Enterprise Case Study: Customer Support Agent

Patterns in action — a real-world architecture combining 6+ patterns



Patterns Used:

Det Shell + Prob Core | Agentic RAG | Agent Handoff | Semantic Cache | Model Routing | HITL Gates

Results

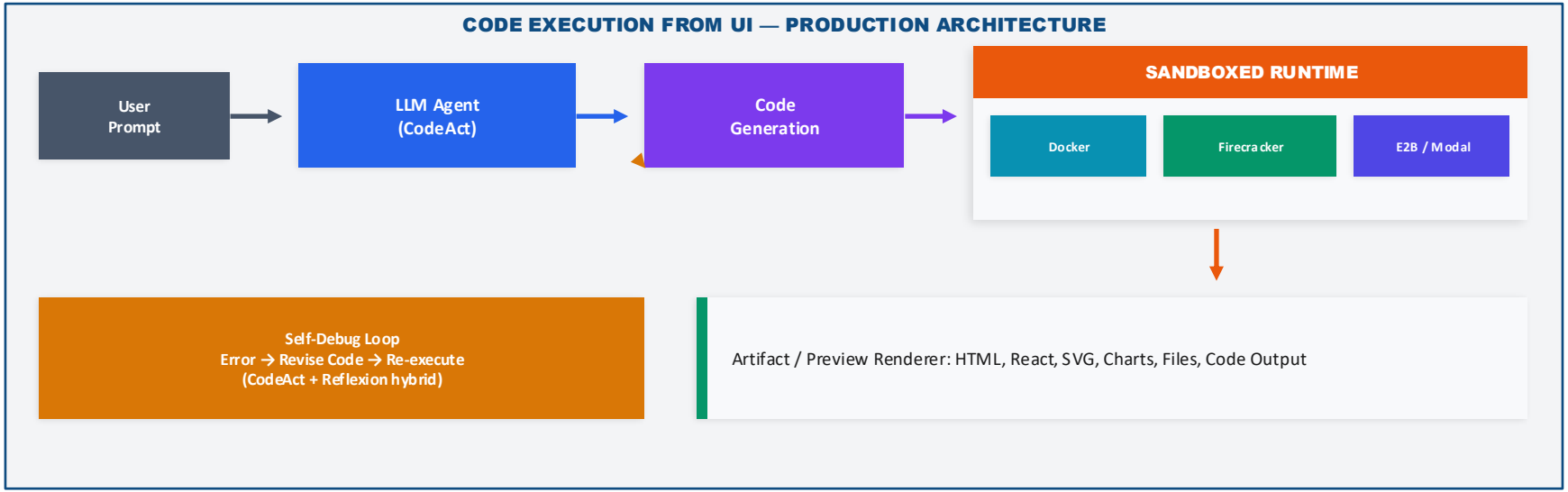
- 70% of tickets auto-resolved (no human needed)
- 45% cost reduction vs pure-premium-model approach
- P95 response time: 8 seconds (was 45 min with humans)

Key Learnings

- Semantic cache alone saved 35% of LLM costs
- Model routing: 60% of queries handled by Haiku-tier
- HITL gates caught 12% of responses that would have been wrong

Case Study: Code Execution from UI

How Claude, ChatGPT Canvas & similar products run code and show previews from a prompt



INPUT: Natural language prompt (e.g., 'build me a dashboard')

OUTPUT: Rendered preview + downloadable artifact + execution log

Patterns Combined

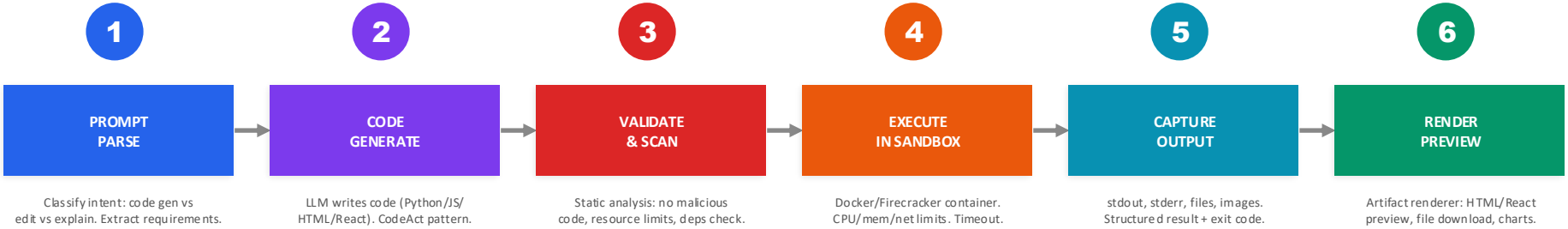
- CodeAct (code as action) + Reflexion (self-debug) + Sandboxing (security)
- Pipeline: Prompt → Generate → Execute → Render → Self-Debug if error

Key Architecture Decisions

- Sandbox isolation: each session = fresh container (no state leakage)
- Network egress: whitelist only (npm, pip) to prevent exfiltration

Code Execution from UI — Deep Architecture

The full pipeline: prompt → code → sandbox → artifact → preview, with security at every layer



↺ Error? Agent reads stderr, modifies code (self-debug), re-executes. Max 3-5 retries.

SANDBOX STARTUP	EXECUTION TIMEOUT	MEMORY LIMIT	NETWORK
<500ms (warm pool)	**30s default, 5min max**	**512MB - 2GB per container**	**Egress whitelist only**

Technology Stack Examples

- Claude Artifacts: React/HTML rendering in iframe sandbox
- ChatGPT Code Interpreter: Python in Firecracker microVM
- Replit Agent: Full container with filesystem + network
- E2B: Cloud sandboxes via API (popular for startups)
- Modal: Serverless container execution for LLM code

Security Layers (Critical)

- Container isolation: each execution = fresh container, destroyed after
- No persistent filesystem: output captured, container discarded
- Network egress: only whitelisted domains (package registries)
- Resource limits: hard caps on CPU, memory, disk, execution time
- Code scanning: block known dangerous patterns before execution

Agent Input/Output Reference Card

Every pattern's I/O contract at a glance — what goes in, what comes out

Pattern	Input Contract	Output Contract
ReAct	Task + tools schema	Final answer + reasoning trace
CodeAct	Task + Python interpreter	Execution result + code + stdout/stderr
ReWOO	Task + tool definitions	Final answer (plan executed without LLM)
Plan & Execute	Complex task description	Step-by-step results + final synthesis
Reflexion	Task + quality rubric	Refined output + improvement log (N iterations)
Supervisor	Task + worker agent registry	Aggregated worker results + supervisor summary
Peer/Debate	Problem + agent roles	Consensus answer + debate transcript
MoA (Ensemble)	Task (sent to N agents)	Best-of-N or synthesized answer
Sub-Agent	Decomposed subtask + scoped context	Subtask result + execution trace
Agent Handoff	Request + conversation context	Response + next-agent handoff signal
Map-Reduce	Large dataset/task split into chunks	Reduced/aggregated final result
RAG	Query + retrieval index	Grounded answer + source citations

Key Takeaways

The essential principles for enterprise agent design

1. Start simple — Single Agent + Tool-Use covers 80% of use cases. Scale complexity only when needed.
2. Hybrid is the norm — Production systems mix deterministic and probabilistic. Design for it.
3. Memory is your cost driver — Token budgets compound in multi-agent systems. Budget before building.
4. SOLID applies — SRP for agents, OCP for tools, DIP via MCP. Software principles work for AI.
5. Design for failure — Circuit breakers, fallback chains, and human escalation are not optional.
6. Observe everything — If you can't trace it, you can't debug it, optimize it, or trust it.
7. Govern costs — Set budgets per agent, per workflow, per team. AI spend grows fast without controls.
8. Use the Canvas — Map every system to the Hybrid Design Canvas. No cell should be blank.

Enterprise Agent Design Patterns

Comprehensive Reference Architecture

Thank you!

Hanuma Pedprolu | M.Tech AI ML(pursuing) | BITS Pilani (WILP)